

Trabajo fin de grado

Análisis de riesgos en los videojuegos online para menores y
simulación de ataques en entorno controlado



Blanca Matas Gavira

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C\Francisco Tomás y Valiente nº 11

UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Análisis de riesgos en los videojuegos online para menores y simulación de ataques en
entorno controlado**

Autor: Blanca Matas Gavira

Tutor: Álvaro Ortigosa Juárez

junio 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual. La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© 17 de junio de 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

Blanca Matas Gavira

Análisis de riesgos en los videojuegos online para menores y
simulación de ataques en entorno controlado

Blanca Matas Gavira

C\ Francisco Tomás y Valiente Nº 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

A mi madre, por ser siempre mi lugar seguro.

La seguridad es un proceso, no un producto.

—Bruce Schneier

AGRADECIMIENTOS

Gracias a mi tutor, por guiarme y ayudarme durante todo el proceso que ha supuesto este trabajo, y también a los profesores de la carrera que me han ayudado a descubrir qué es lo que realmente me gusta de la informática y el propósito por el que quiero dedicarme a ello.

Gracias a mis amigas, por aguantarme, apoyarme y animarme siempre que lo he necesitado, y por darme la fuerza y la felicidad necesarias para no rendirme nunca.

Por último, y no menos importante, gracias a mi familia y, en especial, a mi madre, por creer siempre en mí y acompañarme en cada pasito de esta etapa.

RESUMEN

El crecimiento de la tecnología ha hecho que se incorporen en los videojuegos online accesibles desde navegador funcionalidades propias de aplicaciones web complejas: autenticación de usuarios, perfiles persistentes, comunicaciones en tiempo real, tiendas, recompensas y almacenamiento de información asociada a la cuenta. Este contexto resulta especialmente sensible cuando estas plataformas son utilizadas por menores, ya que elementos visuales como noticias, recompensas o mensajes del sistema pueden influir en la confianza del menor y facilitar acciones engañosas si son manipulados.

Este trabajo analiza patrones de riesgo observables en videojuegos online mediante una metodología pasiva, no intrusiva y limitada a sesiones propias. Se estudian 10 plataformas representativas, clasificando los hallazgos según amenazas relacionadas con contenido dinámico, gestión de sesión, comunicación, micropagos, exposición de datos y endurecimiento técnico. A partir de los patrones más relevantes, se desarrolla un entorno de pruebas aislado en el que se reproducen dos escenarios controlados: una manipulación de contenido dinámico mediante DOM-based XSS y un ataque encadenado que muestra la posible extracción de un token simulado situado en el almacenamiento local. También se proponen y validan mitigaciones basadas en renderizado seguro, listas blancas, construcción controlada del DOM y mejoras en la gestión de sesión, con el objetivo de reducir el impacto de estas debilidades en entornos utilizados por menores.

PALABRAS CLAVE

Videojuegos online, Menores, Ciberseguridad, Análisis pasivo, DOM-based XSS, Gestión de sesión, Prueba de concepto

ABSTRACT

The growth of technology has led browser-accessible online video games to incorporate functionalities typical of complex web applications: user authentication, persistent profiles, real-time communications, shops, rewards, and storage of account-related information. This context is especially sensitive when these platforms are used by minors, since visual elements such as news, rewards, or system messages can influence a minor's trust and facilitate deceptive actions if manipulated.

This work analyses observable risk patterns in online video games through a passive, non-intrusive methodology limited to the author's own sessions. Ten representative platforms are studied, classifying the findings according to threats related to dynamic content, session management, communication, micropayments, data exposure and technical hardening. Based on the most relevant patterns, an isolated testing environment is developed in which two controlled scenarios are reproduced: manipulation of dynamic content through DOM-based XSS and a chained attack that shows the possible extraction of a simulated token stored in local storage. Mitigations based on secure rendering, allowlists, controlled DOM construction and improvements in session management are also proposed and validated, with the aim of reducing the impact of these weaknesses in environments used by minors.

KEYWORDS

Online video games, Minors, Cybersecurity, Passive analysis, DOM-based XSS, Session management, Proof of concept

ÍNDICE

Capítulo 1	Introducción	1
1.1	Motivación y contexto.....	1
1.2	Objetivos	2
1.3	Alcance	2
1.4	Metodología	3
1.5	Estructura	3
Capítulo 2	Marco conceptual y estado del arte	5
2.1	Ciberseguridad en videojuegos online	5
2.2	Amenazas relevantes en videojuegos online	6
2.2.1	Contenido dinámico e inyección en cliente	6
2.2.2	Gestión de sesión y almacenamiento local	7
2.2.3	Comunicaciones en tiempo real	7
2.2.4	Micropagos, recompensas y economías virtuales	8
2.2.5	Exposición de datos e identificadores persistentes	8
2.2.6	Endurecimiento técnico y cabeceras de seguridad.....	8
2.3	Riesgos específicos para menores.....	9
2.4	Referencias y guías de seguridad aplicables	10
2.5	Herramientas de análisis y desarrollo	11
Capítulo 3	Metodología de análisis pasivo	13
3.1	Criterios de selección de juegos y plataformas.....	13
3.2	Entorno e instrumentación	14
3.3	Protocolo de análisis pasivo	15
3.4	Categorías de patrones de riesgo.....	16
3.5	Consideraciones éticas, legales y tratamiento de evidencias	17
Capítulo 4	Resultados del análisis pasivo	19
4.1	Comparativa general de plataformas analizadas	19
4.2	Patrones observados por categoría	21
4.2.1	Contenido dinámico e inyección en cliente	21

4.2.2	Gestión de sesión y almacenamiento local	22
4.2.3	Comunicaciones en tiempo real.....	23
4.2.4	Micropagos, recompensas y economías virtuales	23
4.2.5	Exposición de datos e identificadores persistentes	24
4.2.6	Endurecimiento técnico y cabeceras de seguridad.....	24
4.3	Casos analizados con mayor profundidad.....	24
4.3.1	Venge.io: renderizado dinámico de HTML en cliente	24
4.3.2	Krunker.io: almacenamiento de sesión en cliente	26
Capítulo 5	Diseño e implementación del entorno de pruebas (PoC)	27
5.1	Arquitectura del entorno.....	27
5.2	Diseño general de la aplicación	28
5.3	Decisiones específicas de diseño.....	29
5.3.1	Bloque de noticias	29
5.3.2	Autenticación y generación del token de sesión	30
5.3.3	Modo de renderizado vulnerable/mitigado.....	30
5.4	Reproducción controlada de los patrones de riesgo.....	31
5.4.1	PoC 1: renderizado inseguro de contenido dinámico	31
5.4.2	PoC 2: ataque encadenado sobre sesión	32
5.5	Propuesta y validación de mitigaciones	33
Capítulo 6	Conclusiones y trabajos futuros	37
6.1	Limitaciones del estudio	37
6.2	Interpretación de los resultados	37
6.3	Líneas futuras	38
Capítulo 7	Bibliografía	40
Apéndices	43	
Apéndice A	Montaje del entorno virtual de pruebas	45

LISTAS

LISTA DE FIGURAS

FIGURA 3.1 FLUJO DE INSTRUMENTACIÓN SEGUIDO DURANTE EL ANÁLISIS PASIVO DE LAS PLATAFORMAS SELECCIONADAS	15
FIGURA 4.1 FRAGMENTO DEL COMPONENTE USERNAME DONDE EL VALOR DINÁMICO SE RENDERIZA MEDIANTE V-HTML.....	25
FIGURA 4.2 FUNCIÓN <code>FORMATMARKUP(VALUE)</code> UTILIZADA PARA TRANSFORMAR MARCADO PERSONALIZADO EN ELEMENTOS HTML	25
FIGURA 4.3 CLAVES DE SESIÓN Y USUARIO OBSERVADAS EN <code>LOCALSTORAGE</code> DURANTE UNA SESIÓN PROPIA EN <code>KRUNKER.IO</code>	26
FIGURA 4.4 FUNCIONES AUXILIARES PARA ALMACENAR, ELIMINAR Y RECUPERAR VALORES EN <code>LOCALSTORAGE</code>	26
FIGURA 4.5 CONSULTA DE CLAVES PERSISTENTES ASOCIADAS AL ESTADO DE SESIÓN DEL USUARIO	26
FIGURA 5.1 ARQUITECTURA DEL LABORATORIO CON LAS MÁQUINAS SERVIDOR, CLIENTE LEGÍTIMO Y ATACANTE	27
FIGURA 5.2 INTERFAZ PRINCIPAL DE LA APLICACIÓN PoC CON ESTÉTICA DE VIDEOJUEGO ONLINE Y PANEL DE NOTICIAS DESTACADO	28
FIGURA 5.3 FUNCIÓN DE SANITIZACIÓN PARCIAL APLICADA AL CONTENIDO DINÁMICO ANTES DE SU RENDERIZADO	29
FIGURA 5.4 FUNCIÓN DE MARCADO PERSONALIZADO QUE TRANSFORMA ETIQUETAS DE COLOR EN ELEMENTOS HTML PERMITIDOS.....	29
FIGURA 5.5 FRAGMENTO DE LA GENERACIÓN DEL TOKEN DE SESIÓN SIMULADO MEDIANTE CABECERA, PAYLOAD Y FIRMA.....	30
FIGURA 5.6 TOKEN DE SESIÓN SIMULADO ALMACENADO EN <code>LOCALSTORAGE</code> BAJO LA CLAVE <code>POC_SESSION_TOKEN</code>	30
FIGURA 5.7 SELECTOR INCORPORADO EN LA INTERFAZ	31
FIGURA 5.8 FRAGMENTO DE CÓDIGO QUE CONTROLA EL COMPORTAMIENTO DE RENDERIZADO SEGÚN EL MODO SELECCIONADO	31
FIGURA 5.9 PETICIÓN REALIZADA DESDE LA MÁQUINA ATACANTE PARA MODIFICAR EL CONTENIDO DE NOTICIAS DE LA APLICACIÓN PoC ...	32
FIGURA 5.10 VISUALIZACIÓN DE LA NOTICIA FALSA EN LA INTERFAZ DEL USUARIO LEGÍTIMO TRAS LA MODIFICACIÓN REALIZADA	32
FIGURA 5.11 PETICIÓN DE ACTUALIZACIÓN DE NOTICIAS CON LA CARGA UTILIZADA EN LA PoC DE ATAQUE ENCADENADO SOBRE SESIÓN ...	33
FIGURA 5.12 INTERFAZ DEL CLIENTE LEGÍTIMO CARGANDO LA NOTICIA MODIFICADA UTILIZADA EN LA SEGUNDA PoC	33
FIGURA 5.13 RECEPCIÓN EN LA MÁQUINA ATACANTE DE LA PETICIÓN HTTP CON EL TOKEN DE SESIÓN SIMULADO	33
FIGURA 5.14 VALIDACIÓN DEL MODO VULNERABLE TRAS INTRODUCIR UNA CARGA MALICIOSA.....	34
FIGURA 5.15 VALIDACIÓN DEL MODO MITIGADO, DONDE EL CONTENIDO HTML NO PERMITIDO SE MUESTRA COMO TEXTO LITERAL	34
FIGURA 5.16 FRAGMENTO DE LA MITIGACIÓN APLICADA MEDIANTE LISTA BLANCA Y CONSTRUCCIÓN SEGURA DE NODOS DEL DOM	34
FIGURA A.1 VISTA GENERAL DE LAS MÁQUINAS DEL LABORATORIO EN ORACLE VIRTUALBOX	46
FIGURA A.2 FICHERO DE CONFIGURACIÓN DE NETPLAN DE LA MÁQUINA POC-SERVER	47
FIGURA A.3 VERIFICACIÓN DE CONECTIVIDAD INTERNA Y AUSENCIA DE SALIDA A REDES EXTERNAS EN EL LABORATORIO VIRTUAL	48

LISTA DE TABLAS

TABLA 3.1 CRITERIOS DE SELECCIÓN DE PLATAFORMAS ANALIZADAS.....	14
TABLA 3.2 FASES DEL PROTOCOLO DE ANÁLISIS PASIVO	15
TABLA 3.3 CATEGORÍAS DE PATRONES OBSERVADOS DURANTE EL ANÁLISIS PASIVO	16
TABLA 4.1 CARACTERIZACIÓN GENERAL DE LAS PLATAFORMAS ANALIZADAS.....	20
TABLA 4.2 PATRONES DE RIESGO OBSERVADOS DURANTE EL ANÁLISIS PASIVO	21
TABLA A.1 ROLES ASIGNADOS A LAS MÁQUINAS VIRTUALES DEL LABORATORIO.....	47
TABLA A.2 DIRECCIONAMIENTO IP ASIGNADO A LAS MÁQUINAS DEL LABORATORIO	47

GLOSARIO

TÉRMINO	DEFINICIÓN
ANÁLISIS PASIVO	Observación del comportamiento de una plataforma sin realizar ataques, modificar tráfico ni explotar vulnerabilidades.
CORS	Mecanismo que controla qué orígenes externos pueden realizar peticiones a un recurso web desde el navegador.
CSP	Cabecera de seguridad que limita qué recursos puede cargar o ejecutar una página web.
DOM-BASED XSS	Tipo de Cross-Site Scripting que se produce en el lado cliente cuando datos no confiables se insertan de forma insegura en el DOM.
ENGAGEMENT	Grado de participación, atención o implicación que una plataforma busca mantener en el usuario mediante mecánicas como recompensas, eventos temporales, progresión o actualizaciones frecuentes.
ENTORNO CONTROLADO	Laboratorio aislado donde se reproducen pruebas de forma segura sin afectar a servicios reales ni a usuarios externos.
EXFILTRACIÓN	Extracción no autorizada de información desde una aplicación o sistema hacia un destino externo.
INNERHTML	Propiedad del DOM que permite insertar contenido como HTML dentro de un elemento, por lo que puede ser peligrosa si se usa con datos no confiables.
INVENTARIO	Conjunto de objetos virtuales asociados a la cuenta del jugador, como monedas, armas o recompensas.
JWT	Formato de token compuesto por cabecera, contenido y firma, utilizado habitualmente para transmitir información de autenticación.
LOBBY	Espacio previo a una partida donde los jugadores pueden esperar, agruparse o configurar la sesión de juego.
LOCALSTORAGE	Mecanismo de almacenamiento persistente del navegador accesible desde JavaScript.
MATCHMAKING	Sistema que empareja jugadores o equipos para formar partidas según criterios como disponibilidad, nivel, región, modo de juego u otros parámetros definidos por la plataforma.
MICROPAGO	Compra realizada dentro del videojuego para obtener contenido digital, monedas, objetos o ventajas.
PATRÓN DE RIESGO	Comportamiento observado que puede estar relacionado con una debilidad de seguridad, aunque no se confirme como vulnerabilidad.

PROFILING	Proceso de recogida y análisis de datos de actividad de un usuario para inferir hábitos, preferencias, comportamiento o patrones de uso.
PRUEBA DE CONCEPTO (POC)	Implementación experimental utilizada para demostrar un riesgo o validar una mitigación en un entorno seguro.
QUERY STRING	Parte de una URL situada tras el signo ?, utilizada para enviar parámetros al servidor.
RECOMPENSA VIRTUAL	Elemento ofrecido al jugador como incentivo, como monedas, experiencia, objetos o ventajas temporales.
SANITIZACIÓN	Proceso de limpieza, validación o transformación de una entrada para evitar que contenga contenido peligroso.
SINK	Punto donde una aplicación inserta o procesa datos, especialmente relevante si los interpreta como HTML o código ejecutable.
SKIN	Elemento cosmético que modifica la apariencia de un personaje, arma u objeto del videojuego.
SOURCE	Punto desde el que una aplicación obtiene datos, como una URL, una API, el almacenamiento del navegador o una entrada del usuario.
TOKEN DE SESIÓN	Valor utilizado para identificar una sesión autenticada y asociarla a un usuario concreto.
USUARIO MENOR	Usuario menor de 18 años considerado especialmente relevante en este trabajo por su exposición a plataformas online, mecánicas de recompensa, comunicación con otros jugadores y posibles riesgos de privacidad o engaño.
V-HTML	Directiva de Vue.js que inserta contenido como HTML dentro del DOM.
WEBSOCKET	Canal de comunicación persistente entre cliente y servidor utilizado para intercambiar datos en tiempo real.

Capítulo 1

INTRODUCCIÓN

1.1 Motivación y contexto

El desarrollo continuo de la tecnología ha hecho que, a diferencia de los videojuegos tradicionales, muchos videojuegos actuales incorporen funcionalidades propias de aplicaciones web complejas: autenticación de usuarios, perfiles persistentes, comunicación entre jugadores, tiendas internas y almacenamiento de información asociada a la cuenta. Esta evolución se relaciona con el paso hacia formas de juego en la red, donde la incorporación de otros jugadores a través de Internet transforma la experiencia de juego. Williams describe precisamente el juego online como un formato social relevante, en el que la presencia de competidores y colaboradores introduce una nueva dimensión de interacción entre jugadores [1]. Este tipo de videojuegos tienen un uso especialmente extendido entre menores. En un contexto donde la comunicación se realiza a través de micrófono, chat o sistemas integrados; la socialización se produce a través de Internet, en lugar de mediante un juego local compartido físicamente con otros jugadores. Con este cambio en el paradigma de los videojuegos surgen nuevas preocupaciones: ¿cómo proteger a un menor que, desde temprana edad, maneja con soltura un ordenador de los riesgos que puede encontrar en la red?

Habitualmente, cuando se habla de reducir estos riesgos, una de las primeras respuestas es la concienciación del usuario. Esta educación digital resulta necesaria, pero al tratarse de un público menor de edad y en constante crecimiento, quizá en este tipo de entornos virtuales tan llamativos el foco no debe situarse únicamente en la responsabilidad del usuario, sino también en la seguridad intrínseca que debe ofrecer la plataforma a la que accede y con la que interactúa. UNICEF señala que los videojuegos online pueden ofrecer oportunidades de aprendizaje, colaboración y socialización, pero también pueden exponer a los menores a daños propios de la actividad online [2].

Los videojuegos online presentan superficies de exposición especialmente relevantes desde el punto de vista de la ciberseguridad con un incremento del riesgo cuando son utilizados por un colectivo especialmente vulnerable como son los menores. Por una parte, estos pueden manejar información personal o datos asociados a la identidad digital del menor; en este sentido, la Agencia Española de Protección de Datos recomienda educar a los menores sobre los riesgos para su privacidad y seguridad en el uso de tecnologías conectadas [3]. Por otra

parte, este tipo de plataformas suelen apoyarse en colores llamativos, mensajes destacados, recompensas, eventos temporales y mecánicas de monetización diseñadas para mantener la atención del jugador e incentivar su participación [4]. Si estos elementos fueran manipulados o reproducidos de forma fraudulenta dentro de la propia interfaz, podrían utilizarse para generar confianza, inducir acciones engañosas o aumentar el impacto de determinadas debilidades técnicas.

1.2 Objetivos

El objetivo de este trabajo es analizar patrones de riesgo observables en videojuegos online y reproducir los más relevantes en un entorno controlado, con especial atención a aquellos escenarios que puedan afectar a usuarios menores. Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Identificar las principales amenazas aplicables a videojuegos online accesibles desde el navegador.
- Definir una metodología de análisis pasivo, no intrusiva y éticamente segura para el estudio de plataformas reales.
- Establecer criterios técnicos y contextuales para seleccionar una muestra limitada de videojuegos online representativos.
- Aplicar la metodología definida sobre la muestra seleccionada y recopilar evidencias.
- Clasificar los patrones de riesgo identificados según las amenazas previamente definidas y su posible impacto en usuarios menores.
- Diseñar e implementar un entorno de pruebas aislado que permita reproducir de forma controlada los patrones de riesgo más relevantes.
- Desarrollar pruebas de concepto que muestren el impacto potencial de dichos patrones.
- Proponer y validar medidas de mitigación frente a los escenarios reproducidos.
- Extraer conclusiones y recomendaciones de seguridad aplicables a videojuegos online utilizados por menores.

1.3 Alcance

El estudio está centrado en videojuegos online accesibles desde navegador, con funcionalidades comunes de aplicaciones web interactivas: contenido dinámico, gestión de sesión, elementos de cuenta, recompensas virtuales o comunicación con servidor. Quedan fuera del alcance videojuegos instalados localmente, aplicaciones móviles nativas y cualquier análisis que requiera explotación activa sobre servicios reales. El estudio presta especial atención a los usuarios menores de 18 años, considerados el colectivo objetivo del trabajo. No obstante, las amenazas analizadas pueden afectar igualmente a usuarios de otras edades.

1.4 Metodología

La metodología seguida combina una fase de análisis pasivo y una fase de desarrollo de un entorno de pruebas de forma controlada. En la primera se observan comportamientos visibles desde el navegador, sin realizar ataques, modificación del tráfico, fuerza bruta, evasión de controles ni acceso no autorizado. En la segunda se implementa un entorno propio con máquinas virtuales aisladas, donde se reproducen de forma segura algunos de los patrones observados.

1.5 Estructura

La memoria se estructura en siete capítulos: el presente capítulo introduce la motivación, el problema, el alcance y la metodología general; el capítulo 2 presenta el estado del arte y el marco conceptual; el capítulo 3 describe la metodología de análisis pasivo; el capítulo 4 recoge los resultados obtenidos; el capítulo 5 desarrolla el entorno de pruebas y las pruebas de concepto implementadas junto con sus mitigaciones; el capítulo 6 expone las conclusiones y líneas futuras y, finalmente, el capítulo 7 recoge la bibliografía utilizada.

Capítulo 2

MARCO CONCEPTUAL Y ESTADO DEL ARTE

2.1 Ciberseguridad en videojuegos online

La ciberseguridad tiene actualmente un papel central en el desarrollo de aplicaciones web, ya que estos sistemas exponen múltiples puntos de entrada e interacción con el usuario: formularios, APIs, mecanismos de autenticación, almacenamiento de sesión o comunicaciones cliente-servidor. El tratamiento incorrecto de estas entradas se considera una de las causas principales de vulnerabilidades graves: en el ranking MITRE CWE Top 25 de 2025, las dos debilidades más peligrosas corresponden a problemas de neutralización incorrecta de entradas asociados a Cross-Site Scripting y SQL Injection [5]. El impacto de estas debilidades se basa, tal y como define la organización MITRE, en la capacidad de un atacante para corromper el flujo seguro de datos entre la aplicación y la víctima [6].

Los videojuegos online accesibles desde navegador comparten muchas características técnicas con aplicaciones web con elementos habituales como la actualización dinámica de la interfaz, la gestión de usuarios, la comunicación con servidor y, en muchos casos, la interacción en tiempo real. Entonces, si estos videojuegos comparten tanto con aplicaciones web, ¿no están expuestos a los mismos riesgos típicos de este tipo de entornos? Esta relación no es únicamente teórica: Imperva, compañía especializada en ciberseguridad, señala que la industria del videojuego es un objetivo habitual para los atacantes debido a su volumen económico y a su amplia base de usuarios. Además, identifica las aplicaciones web y APIs como superficies de ataque relevantes en este sector, utilizadas para objetivos como el robo de datos, el acceso no autorizado a sistemas o la interrupción del servicio. De forma especialmente relevante para este trabajo, el mismo informe indica que los sitios de *gaming* sufren ataques de Cross-Site Scripting al doble de ritmo que el conjunto del resto de industrias [7].

Por tanto, aunque el objetivo principal de los videojuegos online sea el entretenimiento, desde el punto de vista técnico, deben ser considerados como sistemas web complejos. Un fallo en la validación de entradas, en el renderizado de contenido dinámico o en la gestión de sesión puede afectar tanto al funcionamiento del juego como a la cuenta del usuario, su información personal, sus objetos virtuales o sus datos de pago. La plataforma de ciberseguridad Cloudflare destaca que los ataques de toma de control de cuenta son una

amenaza común en gaming e *igaming* porque estas cuentas pueden tener valor económico y contener datos personales, incluida información de pago [8].

Esta superficie de exposición se agrava debido a la tendencia actual del consumo rápido, donde lo que estaba de moda hace una semana puede dejar de interesar poco después. Los videojuegos online se ven sometidos a una presión constante por incorporar nuevas mecánicas, recompensas o actualizaciones que mantengan la atención de su público. La relación entre actualizaciones y *engagement* ha sido analizada en videojuegos online, donde se observa que la forma en la que se publican nuevos contenidos puede influir en la participación de los jugadores [9]. Esto puede favorecer a que, en algunos casos, la rapidez de desarrollo o la disponibilidad inmediata de nuevas funcionalidades compita con la revisión adecuada de la seguridad cada vez que se producen cambios, dejando en un segundo plano la necesidad de que la seguridad forme parte del proceso de diseño y desarrollo [10].

2.2 Amenazas relevantes en videojuegos online

Partiendo de la base de que caracterizamos los videojuegos online como sistemas web complejos, este apartado recoge las amenazas que resultan más relevantes para el alcance del trabajo. La selección no pretende cubrir todos los riesgos posibles dentro de la industria de los videojuegos, sino centrarse en aquellos que tienen una relación directa con aplicaciones web de navegador: autenticación, gestión de sesión, renderizado dinámico, comunicación cliente-servidor, lógica de negocio y exposición de datos. Para ordenar las amenazas seleccionadas se toman como apoyo las categorías propuestas por el marco de referencia general OWASP Top 10 2025 [11]. Esta clasificación se complementa con MITRE CWE cuando permite identificar una debilidad técnica concreta [12].

Aunque en el sector gaming también son frecuentes amenazas como los ataques de denegación de servicio, campañas de bots o abuso de credenciales, este trabajo no las desarrolla en profundidad porque afectan principalmente a la disponibilidad o a la automatización a gran escala. El foco se sitúa en amenazas que pueden analizarse desde el lado del cliente web y que posteriormente pueden reproducirse en un entorno de pruebas controlado.

2.2.1 Contenido dinámico e inyección en cliente

Para facilitar la experiencia del usuario, los juegos online de navegador actualizan la interfaz de forma dinámica; algunos contenidos como noticias, mensajes de sistema, nombres de usuario, datos de perfil o avisos

pueden generarse o modificarse durante la ejecución. Si estos datos se insertan en el DOM sin un tratamiento adecuado, puede aparecer una superficie de riesgo asociada a la categoría A05:2025-Injection [13], especialmente en forma de Cross-Site Scripting (XSS) cuando el navegador interpreta contenido no confiable como código ejecutable. Dentro de este grupo, el DOM-based XSS resulta crítico porque el problema se manifiesta en el lado cliente, normalmente cuando datos no confiables son recogidos desde una fuente (*Source*) y posteriormente introducidos en un punto de renderizado inseguro (*Sink*), un patrón catalogado formalmente por MITRE como CWE-79 [6]. En los videojuegos online, esta amenaza no se limita a la ejecución técnica de código, también puede afectar a la confianza del usuario en la interfaz. Un atacante que consiga manipular una zona de noticia o un mensaje de sistema podría inducir acciones engañosas aprovechando elementos propios del juego.

2.2.2 Gestión de sesión y almacenamiento local

La autenticación permite asociar las acciones realizadas en el juego con una cuenta personal del usuario. En los videojuegos online, aparte de que la cuenta representa un mecanismo de acceso, también suele estar vinculada a progreso, inventario, objetos virtuales, historial de compras o interacciones sociales, por lo que la gestión de sesión constituye una superficie especialmente sensible. Uno de los riesgos más relevantes se encuadra en la categoría A07:2025-Authentication Failures de OWASP [14] y aparece cuando los identificadores de sesión o tokens se almacenan en ubicaciones accesibles desde JavaScript, como `localStorage` o `sessionStorage`. Estos mecanismos pueden ser útiles para conservar información en el navegador, pero no proporcionan aislamiento frente a scripts ejecutados dentro del mismo origen. (CWE-922) [15]

2.2.3 Comunicaciones en tiempo real

Las comunicaciones en tiempo real son necesarias en gran parte de los videojuegos online para transmitir eventos de partida, movimiento, estado de salas o interacción entre jugadores. Estas suelen implementarse mediante WebSockets o mecanismos equivalentes, y su seguridad depende de que se utilicen canales cifrados, autenticación adecuada y validación en servidor. El riesgo en esta superficie se presenta de dos formas distintas:

- 1) Exceso de confianza del servidor en los datos enviados por el cliente sin aplicar un control de integridad riguroso, (CWE-602) [16] con lo que un atacante podría forzar comandos arbitrarios o alterar las mecánicas de la plataforma.
- 2) Transmisión de credenciales y tokens de acceso dentro de la propia cadena de consulta (*query string*) utilizada durante el establecimiento de la conexión o *handshake* del WebSocket.

Esta práctica se enmarca en las categorías de gestión insegura de secretos A04:2025-Cryptographic Failures [17] y exposición de credenciales en ubicaciones inseguras A07:2025-Authentication Failures de OWASP [14]. Enviar información sensible en texto legible a través de la URI del flujo en tiempo real provoca que dicha información pueda quedar almacenada en logs, trazas, historial, herramientas de monitorización, capturas HAR o sistemas intermedios. (CWE-201) [18]

2.2.4 Micropagos, recompensas y economías virtuales

Los micropagos, recompensas y economías virtuales forman parte de muchos videojuegos online. La existencia de monedas internas, objetos especiales, pases de temporada o ventajas dentro del juego introduce riesgos adicionales relacionados con la integridad de las transacciones y la validación de datos financieros. El riesgo aparece cuando una operación sensible se delega en la lógica de la interfaz cliente sin una validación adecuada en el servidor, lo que se relaciona con las categorías A06:2025-Insecure Design [19] y A08:2025-Software or Data Integrity Failures de OWASP [20]. En el análisis, esta categoría se analiza únicamente desde una perspectiva pasiva, el objetivo es identificar si durante la navegación normal aparecen patrones relevantes: parámetros de compra, identificadores de objeto, valores de recompensa o eventos iniciados desde el cliente que, en caso de no ser validados posteriormente en servidor, podrían constituir una superficie de riesgo.

2.2.5 Exposición de datos e identificadores persistentes

Otro riesgo clásico asociado a los videojuegos online es el tratamiento de datos personales o identificadores persistentes. Estos datos pueden aparecer en perfiles, telemetría, tokens, sistemas de publicidad, analítica o servicios de terceros. Aunque no todos estos elementos sean datos personales directos por sí solos, permiten perfilar o seguir el comportamiento de un usuario (*profiling*). Esta amenaza se enmarca en la categoría A04:2025-Cryptographic Failures [17] debido a un transporte inseguro o protección inadecuada de la información, y se relaciona con CWE-359 [21]. Esta superficie resulta especialmente relevante porque muchas acciones dentro del juego son persistentes y quedan asociadas a una cuenta. La exposición de datos supone una fuga técnica y una pérdida de privacidad para el usuario dentro de la plataforma.

2.2.6 Endurecimiento técnico y cabeceras de seguridad

El endurecimiento técnico o *hardening* de una aplicación web consiste en aplicar configuraciones destinadas a reducir la superficie de exposición y limitar el impacto de posibles fallos de seguridad. La ausencia

o configuración incorrecta de estos mecanismos se clasifica dentro de la categoría A02:2025-Security Misconfiguration de OWASP [22]. En videojuegos online accesibles desde navegador, este tipo de debilidades puede aumentar el impacto de otras vulnerabilidades presentes en la aplicación. Configuraciones insuficientes relacionadas con el control de recursos cargados por el navegador, protección frente a contenido embebido o la gestión de políticas de seguridad pueden facilitar la explotación o amplificar las consecuencias de otros fallos.

2.3 Riesgos específicos para menores

Una característica importante de los videojuegos online es su presencia extendida entre menores. A nivel europeo, Video Games Europe destaca que los videojuegos forman parte de la vida cotidiana de más de la mitad de los europeos y que millones de niños y jóvenes los utilizan como espacios de entretenimiento y socialización [23]. Otro informe reciente publicado por Ofcom señala que el juego online ocupa un lugar relevante en la vida de los menores: el 61% de los niños de 3 a 17 años juega online [24].

Esta frecuencia de uso en edades tempranas hace que el análisis de riesgos no deba limitarse a la existencia técnica de una amenaza, sino también a la facilidad con la que una debilidad puede camuflarse dentro de una interfaz diseñada para resultar atractiva, rápida y visualmente confiable para el menor. Esta idea resulta especialmente clara en amenazas relacionadas con el contenido dinámico y el renderizado en cliente. Un fallo de tipo XSS o una inserción insegura, aparte de modificar técnicamente la página, serviría para mostrar mensajes fraudulentos con apariencia legítima dentro del propio juego. En una plataforma donde son habituales los avisos de eventos, recompensas temporales, *skins*, armas u objetos exclusivos, un mensaje manipulado puede confundirse fácilmente con una notificación real del sistema. El principal vector de riesgo de esta amenaza para los menores es la manipulación de la confianza del usuario. A diferencia de lo que podría ocurrir en un entorno bancario o corporativo, donde el usuario tiende a mantener una actitud más alerta, en un videojuego la frontera entre una mecánica propia de la plataforma y un engaño puede resultar mucho más difusa. Esto es especialmente sensible en menores, que pueden actuar con menor cautela ante mensajes que prometen ventajas, recompensas o acceso a contenido exclusivo.

También adquieren especial importancia las amenazas relacionadas con la gestión de sesión y la cuenta del usuario. Aunque normalmente se relaciona la cuenta como un mecanismo de acceso, en un videojuego online ésta puede estar asociada al progreso del jugador, inventario, objetos virtuales, historial de compras o

relaciones sociales dentro de la plataforma. Un compromiso de sesión podría afectar a elementos que tienen valor funcional, económico o emocional para el menor.

Otro aspecto importante es la interacción social en los videojuegos online, que normalmente incorporan chats, listas de amigos, salas públicas o sistemas de invitación. Estas funcionalidades aumentan la superficie de exposición frente a mensajes engañosos, acoso, suplantación o intentos de redirección hacia espacios externos al juego. Ofcom indica que, entre los niños de 8 a 17 años que juegan online, el 74% utiliza funciones de chat dentro del juego y el 31% afirma hablar con personas que no conoce mientras juega [24]. Cuando estas comunicaciones se producen dentro de una plataforma que el menor percibe como un entorno de ocio, el nivel de alerta puede ser más bajo que en otros contextos.

En los videojuegos online, las mecánicas de monetización y micropagos añaden un incentivo social relevante para los más pequeños. La promesa de mejorar rápidamente en el juego o conseguir contenido exclusivo que otros jugadores no tienen puede elevar el estatus del usuario dentro de su comunidad. En una edad en la que encajar y formar parte del grupo puede tener mucho peso, este tipo de recompensas puede hacer que el menor actúe con menos cautela ante mensajes o enlaces que prometen ventajas dentro del juego, aunque estos puedan ser fraudulentos.

También deben considerarse los riesgos relacionados con la exposición de datos y la configuración de seguridad de la propia aplicación. En videojuegos online, la actividad del usuario suele quedar asociada a identificadores persistentes, telemetría, inventarios o interacciones sociales, por lo que una exposición de datos afecta a la privacidad y al seguimiento del comportamiento digital del menor dentro de la plataforma. La AEPD advierte que determinados patrones de diseño pueden incrementar la recogida o inferencia de datos personales, incluidos hábitos, preferencias, horarios de conexión o comportamiento del usuario [25]. Por otro lado, la ausencia de políticas de endurecimiento adecuadas no constituye de por sí sola un ataque dirigido a menores, pero sí puede aumentar el impacto de otras amenazas.

2.4 Referencias y guías de seguridad aplicables

Para el desarrollo del trabajo se utilizan varias guías de seguridad como apoyo conceptual y técnico. OWASP Top 10 2025 [11] se toma como marco general para ordenar riesgos propios de aplicaciones web, mientras que MITRE CWE [12] se emplea de forma puntual para identificar debilidades técnicas concretas. Estas

dos guías son las que se utilizan como referencia y permiten en el apartado previo relacionar las amenazas seleccionadas con categorías reconocidas en seguridad web.

En secciones donde se considera que el análisis requiere mayor profundidad, se tienen en cuenta guías específicas de OWASP. Para el tratamiento de XSS se consideran la Cross-Site Scripting Prevention Cheat Sheet [26] y la DOM-based XSS Prevention Cheat Sheet [27], que describen riesgos asociados a la inserción de contenido no confiable y sus mitigaciones. Para la gestión de sesión se utiliza como referencia la Session Management Cheat Sheet [28], concretamente lo relacionado con la protección de identificadores de sesión frente a acceso desde JavaScript. También se considera la OWASP Web Security Testing Guide [29] para contextualizar áreas como validación de entradas, lógica de negocio, WebSockets, almacenamiento en navegador y APIs.

2.5 Herramientas de análisis y desarrollo

Durante el desarrollo del trabajo se utilizaron distintas herramientas de apoyo para observar el comportamiento de las plataformas analizadas y construir posteriormente el entorno de pruebas. Para la observación del cliente web se emplearon principalmente Chrome DevTools [30] y Burp Suite [31], orientadas a inspeccionar recursos cargados, peticiones, cabeceras, almacenamiento del navegador y tráfico generado durante sesiones propias. Para el análisis local de evidencias se utilizaron Wireshark [32], tshark [33], tcpdump [34] y jq [35], permitiendo revisar capturas, ficheros HAR y estructuras JSON de manera más efectiva. Para la construcción del entorno de pruebas se utilizaron Oracle VirtualBox [36] y Node.js [37], empleados respectivamente para desplegar el laboratorio aislado y desarrollar la aplicación web de prueba. Esta aplicación se implementó mediante tecnologías web estándar, utilizando HTML, CSS y JavaScript en el lado cliente, junto con Node.js en el lado servidor.

Capítulo 3

METODOLOGÍA DE ANÁLISIS PASIVO

El objetivo de este análisis es identificar patrones de riesgo observables desde el lado del cliente que puedan ser relevantes para el trabajo presente. Las plataformas analizadas se utilizaron como lo haría un usuario legítimo: accediendo a la página web, observando la carga inicial de recursos, revisando el comportamiento de la interfaz, entrando en partidas o salas disponibles, consultando secciones de perfil, noticias o tienda. Los patrones de riesgo identificados se utilizaron como base para seleccionar los escenarios reproducidos posteriormente en el entorno de pruebas descrito en el capítulo 5. Esta separación entre observación pasiva y experimentación controlada resulta fundamental para el enfoque del trabajo puesto que permitirá que el estudio sea apoyado y respaldado por comportamientos reales observables de videojuegos, pero sin pruebas intrusivas sobre sistemas ajenos, dejando estas reservadas para la fase experimental. Toda la información registrada pertenece a sesiones propias controladas.

3.1 Criterios de selección de juegos y plataformas

La selección de los casos analizados se realizó atendiendo a criterios técnicos y contextuales. Se consideran especialmente relevantes las plataformas que permiten observar de forma directa elementos típicos de aplicaciones web modernas: carga de recursos JavaScript, almacenamiento local, comunicación directa entre cliente y servidor, gestión e identificación de sesión, uso de APIs y contenido dinámico. Desde el punto de vista contextual, se prestó especial atención a juegos que puedan interesar a menores: estética visual, partidas rápidas y frenéticas, mecánicas competitivas, sistema de recompensas llamativo y que sean de actualidad y uso habitual. No se pretende cubrir de forma exhaustiva la industria del videojuego ni comparar todos los tipos de plataformas existentes puesto que el objetivo es obtener casos suficientemente representativos para observar patrones técnicos coherentes con el marco conceptual. La tabla 3.1. recoge los principales criterios utilizados y su justificación dentro del trabajo.

Criterio	Justificación dentro del trabajo
Acceso desde navegador	Permite realizar análisis pasivo mediante herramientas del propio navegador sin instalación de cliente nativo.
Comunicación cliente-servidor observable	Facilita el estudio de peticiones, respuestas, cabeceras, APIs y flujos de sesión.
Contenido dinámico en la interfaz	Permite analizar superficies relacionadas con renderizado inseguro y DOM-based XSS.
Gestión de cuenta, sesión o perfil	Permite estudiar tokens, identificadores persistentes, almacenamiento local o cookies.
Interacción social o comunicación en tiempo real	Facilita observar patrones asociados a chats, salas, lobbies, WebSockets o mensajes entre usuarios.
Atractivo visual y dinámicas de recompensa	Permite seleccionar juegos con estímulos llamativos, progresión rápida o elementos que puedan resultar especialmente atractivos para usuarios menores.
Mecánicas de monetización	Permite verificar si la transmisión de parámetros críticos del negocio se delega de forma insegura en el navegador del usuario.
Popularidad o uso reconocible	Aumenta la relevancia contextual del caso dentro del estudio.

Tabla 3.1 Criterios de selección de plataformas analizadas

Estos criterios sirvieron para acotar el conjunto inicial de plataformas y seleccionar aquellas que ofrecen mayor interés desde el punto de vista del análisis pasivo.

3.2 Entorno e instrumentación

El análisis pasivo se realizó únicamente utilizando cuentas propias o sesiones de invitado, disponibles para cualquier usuario de la plataforma. Para obtener resultados lo más limpios posibles cada sesión se grababa por fecha y hora de manera independiente, limpiando previamente todos los datos pertenecientes a sesiones anteriores.

La instrumentación se organizó en torno a la captura de evidencias generadas durante la navegación normal por cada videojuego online. Antes de iniciar cada sesión se activó el registro de red del navegador para poder exportar las peticiones en formato HAR. Al mismo tiempo también se utilizó Burp Suite como proxy local de observación para revisar las solicitudes y respuestas generadas por el navegador, siempre limitando su uso a la inspección de tráfico propio. Por otra parte, se realizaron capturas locales de tráfico mediante tcpdump,

Wireshark y tshark. Estas capturas sirvieron para contrastar la información observada desde el navegador y analizar los patrones de comunicación. Una vez obtenidas las evidencias, el análisis se realizó de forma local. Los ficheros HAR, JSON o pcap se revisaron mediante herramientas como jq, Wireshark o tshark con el objetivo de realizar un filtrado eficiente de la información previamente capturada. La instrumentación puede resumirse en el flujo representado en la Figura 3.1.

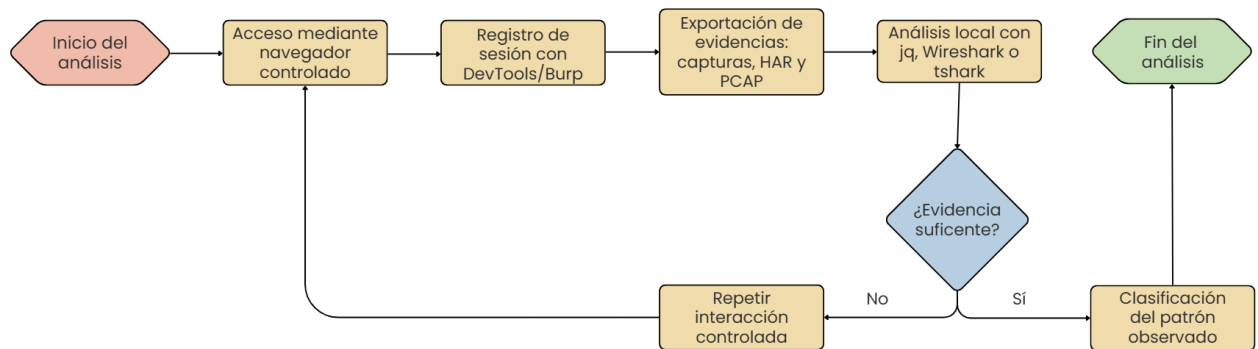


Figura 3.1 Flujo de instrumentación seguido durante el análisis pasivo de las plataformas seleccionadas

3.3 Protocolo de análisis pasivo

El análisis de cada plataforma siguió un protocolo común para mantener la mayor homogeneidad posible entre los casos revisados, limitado siempre a comportamientos observables desde el cliente web. Las fases aplicadas se resumen en la tabla 3.2.

Fase	Actividad realizada	Evidencia generada
Carga inicial	Acceso a la plataforma y observación de recursos cargados	Recursos JS, HTML, dominios y cabeceras iniciales
Interacción básica	Navegación por perfil, lobby, partida, noticias, tienda o recompensas	Peticiones generadas por uso legítimo
Revisión de almacenamiento	Consulta de cookies y almacenamiento del navegador	Identificadores, tokens, claves locales o datos persistentes
Análisis de tráfico propio	Registro de HAR o pcap durante sesiones controladas	Peticiones, respuestas, WebSockets y cabeceras
Filtrado local	Revisión de evidencias capturadas mediante herramientas locales	URLs, parámetros, cookies, campos JSON y dominios
Revisión de flujo de datos	Relación entre datos recibidos por API y puntos de inserción en la interfaz	Posibles sources, sinks y patrones compatibles con DOM-based XSS
Clasificación	Relación del patrón observado con las categorías del capítulo 2	Patrón de riesgo, limitación y relevancia para el trabajo

Tabla 3.2 Fases del protocolo de análisis pasivo

3.4 Categorías de patrones de riesgo

Las categorías revisadas durante el análisis se definieron a partir de las amenazas del marco conceptual del capítulo 2. Estas categorías se presentan como un conjunto de patrones observables que podrían suponer superficies de exposición relevantes, no se califican como vulnerabilidades confirmadas debido a que el fin del análisis nunca ha sido la explotación de las plataformas reales.

Se consideró que un patrón era relevante cuando existía una combinación suficiente de elementos técnicos asociados a una amenaza concreta. Por ejemplo, la presencia de sinks de inserción HTML, como `innerHTML` o `insertAdjacentHTML` junto con datos dinámicos procedentes de una API, puede indicar una posible superficie de DOM-based XSS pero como no se confirma que un dato controlable alcanza ese sink sin sanitización, el resultado se trata como un patrón de riesgo. A partir de este criterio, la tabla 3.3 resume las principales categorías de patrones revisadas durante el análisis pasivo, junto con los elementos observables desde el lado del cliente asociados a cada una y su relación con el alcance del trabajo.

Categoría	Elementos observados	Relación con el trabajo
Contenido dinámico e inyección en cliente	Uso de <code>innerHTML</code> , <code>insertAdjacentHTML</code> , renderizado de datos recibidos por API, zonas de noticias y mensajes del sistema	Relacionado con DOM-based XSS y manipulación de interfaz
Gestión de sesión y almacenamiento local	Tokens, identificadores de sesión, cookies, <code>localStorage</code> , <code>sessionStorage</code> , JWT y otros valores persistentes	Relacionado con exposición de sesión y posible encadenamiento con XSS
Comunicaciones en tiempo real	WebSockets, handshakes, parámetros en URL, mensajes, lobbies, salas y eventos de partida	Relacionado con validación de mensajes y exposición de datos en canales persistentes
Micropagos, recompensas y economías virtuales	Flujos de tienda, parámetros de compra, objetos, monedas, importes y proveedores de pago	Relacionado con lógica de negocio y confianza en datos del cliente
Exposición de datos e identificadores persistentes	Telemetría, dominios de terceros, identificadores de usuario, datos de perfil, analítica y publicidad	Relacionado con perfilado, privacidad y exposición de datos
Endurecimiento técnico y cabeceras de seguridad	CSP, CORS, HSTS, Referrer-Policy, X-Frame-Options y X-Content-Type-Options	Relacionado con configuración defensiva y reducción del impacto de otras amenazas

Tabla 3.3 Categorías de patrones observados durante el análisis pasivo

Esta clasificación permite ordenar los elementos observados para que posteriormente fuesen evaluados en función de su relevancia técnica, su posible impacto en menores y su adecuación para ser reproducidos de forma segura en el laboratorio.

3.5 Consideraciones éticas, legales y tratamiento de evidencias

El análisis de videojuegos online cubierto en el trabajo presente plantea límites éticos y legales importantes ya que se trabaja sobre servicios de terceros y en un contexto en el que pueden existir usuarios menores. Por este motivo todo el análisis se diseñó para evitar cualquier tipo de acción que pudiera afectar a la disponibilidad, integridad o confidencialidad de las plataformas utilizadas.

Como se ha mencionado a lo largo del documento, todas las pruebas realizadas se limitaron a la observación del comportamiento visible desde el cliente durante sesiones propias. No se han realizado en ninguna circunstancia intentos de acceso no autorizado, fuerza bruta, escaneo agresivo, manipulación de *payloads* y alteración o envío de datos tanto a otros usuarios como al servidor pertinente. En cuanto al tratamiento de las evidencias encontradas como capturas de pantalla, ficheros HAR, URLs o identificadores de sesión, se tuvo especial cuidado para evitar la exposición de información sensible, utilizando siempre datos propios. En la memoria no se incorporan tokens completos, direcciones de correo electrónico completas, identificadores de sesión reutilizables, JWT completos, claves o direcciones IP personales. En los casos donde fue necesario mostrar evidencias propias, los valores sensibles se muestran anonimizados.

Capítulo 4

RESULTADOS DEL ANÁLISIS PASIVO

El análisis se aplicó sobre 10 videojuegos online accesibles desde navegador: **Krunker.io**, **Cryzen.io**, **Venge.io**, **SmashKarts.io**, **ZombsRoyale.io**, **ShellShock.io**, **WarBrokers.io**, **Surviv.io**, **Bloxd.io** y **BattleDudes.io**. La selección de estos videojuegos se realizó en base a los criterios definidos en el capítulo anterior.

No todos los juegos ofrecieron el mismo nivel de evidencia observable. Por este motivo, el capítulo se estructura en dos niveles: una primera comparativa general de todos para tener una visión amplia y representativa de la muestra y un segundo nivel donde se relacionan los distintos videojuegos con las amenazas definidas en el capítulo 2, dando más nivel de detalle a aquellos juegos que poseen patrones de riesgo más relevantes para los objetivos del trabajo.

4.1 Comparativa general de plataformas analizadas

La siguiente comparativa resume los principales elementos observados durante el análisis pasivo. Estas tablas no son un ranking de seguridad ni una clasificación formal de severidad. El objetivo es hacer una comparación cualitativa de patrones visibles desde el cliente. Para facilitar la lectura, la comparativa se divide en dos tablas: la tabla 4.1. recoge la caracterización general de las plataformas analizadas y la tabla 4.2. se centra en los patrones de riesgo observados y su relación con las amenazas definidas. Los niveles utilizados en las tablas son los siguientes:

- Alto: patrón observado con evidencia clara y relación directa con el trabajo.
- Medio: patrón observado parcialmente o con relevancia contextual.
- Bajo: patrón poco relevante.
- No concluyente: no se obtuvo evidencia suficiente para clasificarlo.
- No observado: no se identificó el patrón durante el análisis.

Plataforma	Tipo / arquitectura visible	Acceso / cuenta	Tiempo real	Tienda / recompensas	Terceros / privacidad
Krunker.io	WebGL/WASM, bundles JS, arquitectura distribuida	Cuenta/invitado	WebSocket	Tienda, recompensas	Medio
Venge.io	PlayCanvas, Vue, WebGL/WASM	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Medio
Cryzen.io	SPA con bundles Vite/Vue, APIs regionales	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Medio/alto
SmashKarts.io	Unity WebGL/WASM, Photon	Cuenta/invitado	WebSocket/Photon	Tienda, recompensas, Xsolla	Alto
ZombsRoyale.io	Canvas/WebGL, backend M28	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Alto
ShellShock.io	WebGL/Babylon.js, scripts externos	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Alto
WarBrokers.io	Unity WebGL, jQuery, servidores regionales	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Medio
Survev.io	Canvas/WebGL, scripts propios y ads	Cuenta/invitado	WebSocket	Recompensas	Alto
Bloxd.io	WebGL/WASM, motor 3D, Firebase/ads	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Alto
BattleDudes.io	Motor propio, WebWorkers, microservicios	Cuenta/invitado	WebSocket	Tienda, recompensas, Xsolla	Medio

Tabla 4.1 Caracterización general de las plataformas analizadas

Esta primera tabla permite contextualizar la muestra analizada. Todas las plataformas presentan características muy similares, propias de aplicaciones web complejas, siendo la mayor diferencia entre estas la arquitectura.

Plataforma	Renderizado HTML dinámico	Sesión / almacenamiento	WebSocket / parámetros	Hardening / cabeceras	Relevancia para PoC
Krunker.io	Medio/Bajo	Alto	Alto	Parcial/débil	Alta, PoC 2
Venge.io	Alto	Medio	Bajo/medio	Parcial/débil	Alta, PoC 1
Cryzen.io	Alto	Alto	Alto	Parcial/débil	Alta, apoyo
SmashKarts.io	Bajo	Medio	Medio	Débil/parcial	Media
ZombsRoyale.io	Bajo	Bajo/medio	Bajo	Débil/parcial	Baja/media
ShellShock.io	Bajo	Medio	Bajo/medio	Débil/parcial	Baja/media
WarBrokers.io	Medio	Medio	Bajo/medio	Débil/parcial	Media
Surviv.io	Medio	Bajo/medio	Bajo/medio	Débil/parcial	Media
Bloxd.io	Bajo	Bajo/medio	Bajo/medio	Débil/parcial	Media/baja
BattleDudes.io	Bajo	Medio	Bajo/medio	Débil/parcial	Media/baja

Tabla 4.2 Patrones de riesgo observados durante el análisis pasivo

Esta segunda tabla permite agrupar los patrones de riesgo observados y mapearlos con las amenazas definidas anteriormente.

4.2 Patrones observados por categoría

4.2.1 Contenido dinámico e inyección en cliente

En los videojuegos online analizados la interfaz se actualizaba de forma constante para mostrar nombres de usuarios, mensajes del sistema, información de la partida, chats, tablas, entre otros. Este comportamiento es esperado en aplicaciones interactivas, pero puede suponer un vector de riesgo cuando los datos se insertan mediante mecanismos que interpretan cadenas como HTML. El caso más representativo de esto fue **Venge.io**. En su cliente se identificó un componente de nombre usuario (`username`) que utiliza `v-html` para insertar en la interfaz el resultado del método `formatMarkup(value)`. `value` es una propiedad dinámica del componente y el método transforma marcas personalizadas en etiquetas HTML, como formatos de color o efectos visuales.

Cryzen.io también presentó patrones relevantes en esta categoría. En el cliente se observaron notificaciones que tenían propiedades como el título y el texto que se insertaban mediante `innerHTML`. Este caso es especialmente interesante por la manipulación de las notificaciones en la interfaz del usuario, ya que

son elementos de confianza dentro de un videojuego: avisos, eventos, recompensas o mensajes del sistema. En **WarBrokers.io** se identificaron funciones que solicitan datos dinámicos como el número de jugadores conectados y posteriormente insertan la respuesta en el DOM mediante `innerHTML`, este patrón depende completamente de la confianza en la respuesta del servidor. No se trata de una entrada de usuario directa, pero sí representa una práctica menos segura que usar `textContent` cuando el dato esperado es texto.

En **Surveyv.io** se observaron múltiples asignaciones mediante `innerHTML` en elementos como mensajes de recogida de objetos, registro de eliminaciones, mensajes de interacción o datos de armamento. Muchos de estos valores parecen proceder del estado interno del juego o de diccionarios locales. Aun así, el uso constante de `innerHTML` para textos de interfaz es un patrón que se podría mejorar. En **SmashKarts.io**, **ZombsRoyale.io**, **Bloxd.io** y **BattleDudes.io** se observó el uso de `innerHTML` o mecanismos equivalentes normalmente asociados a publicidad, librerías, infraestructura de terceros o código ofuscado por lo que se clasificaron con un nivel bajo.

4.2.2 Gestión de sesión y almacenamiento local

En las plataformas analizadas, todas tenían la posibilidad de registrarse e iniciar sesión para tener perfil propio, este servía para crear una identidad dentro del juego, pudiendo asociar al perfil distintos elementos como nivel, logros o armas personalizadas y poder comprar y gestionar los objetos. Esta opción enriquece la experiencia del usuario, pero si no se gestionan bien tanto el punto de acceso como los identificadores asociados puede llegar a suponer un problema de seguridad. El caso más claro fue **Krunker.io**, en la sesión se observaron claves en `localStorage` como `krunker_token`, `krunker_username` y `krunker_id`. El peligro viene cuando en el código cliente se encontraron funciones auxiliares para guardar, borrar y recuperar valores mediante `localStorage`, así como comprobaciones que consultan dichas claves para determinar el estado del usuario. Este patrón resulta relevante porque cualquier script ejecutado en el mismo origen podría leer o manipular estos valores si existiera una inyección en cliente.

De manera similar, en **Cryzen.io** aparecían valores como `access_token` y `refresh_token` en almacenamiento local. En este caso también se podrían llegar a obtener valores sensibles sobre la identificación del usuario si no se implementasen medidas de seguridad adicionales, e incluso aunque no fuese posible la inyección de código seguiría siendo un patrón de diseño inseguro. En otras plataformas, como **ShellShock.io**, **WarBrokers.io**, **ZombsRoyale.io**, **Bloxd.io** o **BattleDudes.io**, el almacenamiento local estaba más relacionado con preferencias, configuración del juego, identificadores publicitarios, consentimiento o datos de telemetría.

Estos elementos no tienen el mismo peso que un token de sesión, pero reflejan una tendencia común: gran parte del estado de la aplicación se conserva en el navegador y puede quedar expuesto a terceros.

4.2.3 Comunicaciones en tiempo real

Los juegos analizados utilizan comunicaciones en tiempo real para sincronizar partidas, salas, *matchmaking* o chat. Estas comunicaciones estaban implementadas con WebSockets o canales persistentes equivalentes. En **Krunker.io** se observó una conexión WebSocket cuya URL incluía parámetros como `clientKey` y `clientUID` y un parámetro `at` con estructura de token. Aunque la conexión se realiza mediante `wss://` lo que protege el canal mediante TLS, la inclusión de credenciales o identificadores sensibles en la query string es una práctica de riesgo puesto que los valores pueden quedar registrados en distintos puntos, como historiales, logs o entornos intermedios.

En **Cryzen.io** también se observaron conexiones WebSocket con parámetros como `sessionId` y `_authToken` expuestos directamente en la URL de la petición. La captura del tráfico en tiempo real demostró que el parámetro `_authToken` transportaba un JSON Web Token (JWT) completo que al ser decodificado con Base64 expuso en texto plano datos sensibles del perfil propio como el correo electrónico, el nombre real y el enlace a la foto de perfil de Google representando nuevamente un riesgo en caso de que estos valores puedan ser exfiltrados. En **Venge.io**, **ShellShock.io**, **WarBrokers.io**, **Survev.io**, **Bloxd.io** y **BattleDudes.io** los parámetros visibles correspondían únicamente a identificadores de sala, región, versión del cliente o servidor de partida por lo que no representan el mismo nivel de amenaza que los casos anteriores.

4.2.4 Micropagos, recompensas y economías virtuales

La mayoría de los juegos analizados incorporan algún tipo de economía virtual, tienda, sistema de recompensas, o monedas internas. En las plataformas se observaron flujos de compra normalmente asociados a proveedores externos, especialmente Xsolla, Google Pay, Apple Pay, PayPal, Bizum o tarjetas bancarias. Dentro del alcance de este trabajo no se realizaron compras reales ni modificación de productos. Aun así, se analizó este flujo puesto que muchas operaciones de compra comenzaban desde el cliente web, donde aparecen valores críticos de negocio, como precio, moneda o estado de recompensa y si esta información no es validada por el servidor posteriormente, supone un patrón de riesgo.

El caso más interesante fue **SmashKarts.io**, donde había eventos de anuncios recompensados y comunicación entre JavaScript y Unity mediante `SendMessage`. Esto resulta relevante si la validación de

recompensas dependiera únicamente del evento comunicado desde el cliente. En **ZombsRoyale.io** se observó una función que inicia compras mediante una ruta construida con un parámetro variable sin validación contra listas blancas en el lado del cliente, lo cual podría indicar una superficie de manipulación de rutas o de lógica de negocio, aunque no se confirmó explotación al quedar fuera del alcance del análisis pasivo.

4.2.5 Exposición de datos e identificadores persistentes

En casi todas las plataformas se observaron integraciones con servicios externos. Aunque en la mayoría de estos servicios el principal interés es comercial y de monetización, aumentan la superficie de ataque y la implicación de terceros. Los casos más evidentes fueron **ZombsRoyale.io**, **ShellShock.io**, **SmashKarts.io**, **Bloxd.io** y **Surviv.io**, especialmente por la integración de redes publicitarias, sistemas de consentimiento, analítica, anuncios o Firebase. El riesgo en estos juegos no era la exposición directa de credenciales, sino más bien la acumulación de identificadores persistentes, cookies de terceros y datos de telemetría que puedan afectar a la privacidad, al perfilado de comportamiento y a la correlación de actividad entre sesiones o servicios.

4.2.6 Endurecimiento técnico y cabeceras de seguridad

En los videojuegos seleccionados se analizaron cabeceras como *Content-Security-Policy*, *X-Frame-Options*, *Strict-Transport-Security*, *X-Content-Type-Options*, *Referrer-Policy* y políticas CORS. En varias plataformas se clasificaron las configuraciones como parciales o débiles, especialmente por ausencia de CSP, ausencia de mecanismos anti-clickjacking o políticas CORS demasiado amplias. Este tipo de configuraciones puede aumentar el impacto de otras amenazas como una inyección en cliente o compromiso de cuenta. Por lo que es necesario revisar estos mecanismos para reducir vectores de riesgo con una configuración defensiva restrictiva.

4.3 Casos analizados con mayor profundidad

4.3.1 Venge.io: renderizado dinámico de HTML en cliente

Venge.io fue seleccionado como caso principal para la primera prueba de concepto relacionada con renderizado inseguro de contenido dinámico. Esto se debe al patrón de riesgo que posee en el componente de nombre de usuario (`username`). Este recibe distintos valores dinámicos mediante propiedades como `value`, `emoji`, `verified`, `link` o `clan`. El elemento más destacado desde el punto de vista de seguridad es el nombre del jugador, al ser un elemento controlado por el usuario, ya que el valor recibido en `value` se procesa

mediante la función `formatMarkup(value)` y posteriormente se inserta en la interfaz mediante la directiva `v-html`. La Figura 4.1 muestra este patrón de riesgo en el código del videojuego.

La función `formatMarkup()` permite interpretar un marcado propio de la plataforma. Transforma etiquetas personalizadas como `[color="..."]` y `[/color]` en elementos HTML `<span>` con estilos de color y convierte marcas como `[rainbow]` y `[/rainbow]` en elementos visuales asociados a una clase CSS específica, la Figura 4.2 muestra esta función.

```
Vue.component("username", {
  props: ["value", "emoji", "verified", "link", "clan"],
  data: function() {
    return {}
  },
  methods: {
    getImage: Utils.getImage,
    formatMarkup: function(e) {
      return e && e.replace && !e.includes("[rainbow]") ? e.rep
      //...
    },
  },
  template: '... v-html="formatMarkup(value)"></span></div></a>',
  mounted: function() {}
})
```

Figura 4.1 Fragmento del componente `username` donde el valor dinámico se renderiza mediante `v-html`

```
formatMarkup: function(e) {
  return e && e.replace && !e.includes("[rainbow]")
    ? e.replace(/\[color="(.*?)"\]/g, '<span style="color:$1">')
      .replace(/\[/color]/g, '</span>')
      .replace(/\[/g, "")
    : e && e.replace && e.includes("[rainbow]")
    ? e.replace(/\[rainbow]/g, '<span class="rainbow">[')
      .replace(/\[/rainbow]/g, "]/>")
    : e
}
```

Figura 4.2 Función `formatMarkup(value)` utilizada para transformar marcado personalizado en elementos HTML

El valor original deja de tratarse únicamente como texto plano y pasa a convertirse en contenido HTML interpretado por el navegador. Este comportamiento resulta peligroso porque `v-html` desactiva el escape automático que normalmente aplica el *framework* Vue, insertando el contenido generado por la función como HTML dentro del DOM.

Dentro de los videojuegos online, este patrón resulta especialmente sensible porque afecta a un elemento de identidad visible para otros usuarios: el nombre de jugador. Normalmente, los nombres de usuario aparecen en perfiles, tablas, partidas, mensajes o listados de jugadores, por lo que cualquier manipulación de su representación visual podría afectar tanto al usuario propio como a jugadores externos que entren en contacto con el usuario afectado. Además, el uso de marcado personalizado muestra cómo una funcionalidad pensada para mejorar la experiencia del usuario puede convertirse en una superficie de riesgo.

La primera prueba de concepto del capítulo 5 se inspira en este patrón, pero no pretende reproducir exactamente el código de **Venge.io**. En el laboratorio se utiliza `innerHTML` ya que permite representar de forma directa el mismo problema al insertar contenido dinámico como HTML en lugar de mostrarlo como texto plano debido al uso de marcado personalizado.

4.3.2 Kruner.io: almacenamiento de sesión en cliente

Kruner.io es utilizado para la segunda prueba de concepto relacionada con exposición de sesión en cliente y ataque encadenado. Durante el análisis se observaron claves persistentes en `localStorage` asociadas a la sesión y al usuario, entre ellas `kruner_token`, `kruner_username` y `kruner_id`. La Figura 4.3 muestra estos valores asociados a una sesión propia y anonimizados.

kruner_id	4 [REDACTED]
kruner_username	b [REDACTED]
kruner_token	P [REDACTED]

Figura 4.3 Claves de sesión y usuario observadas en `localStorage` durante una sesión propia en Kruner.io

Por otro lado, en el código del cliente se identificaron funciones auxiliares utilizadas para guardar, eliminar y recuperar valores persistentes. Se localizaron funciones como `saveVal`, `deleteVal` y `getSavedVal`, implementadas internamente mediante `localStorage.setItem()`, `localStorage.removeItem()` y `localStorage.getItem()`. Esto confirma que el almacenamiento local forma parte de la lógica de persistencia utilizada por la plataforma. La Figura 4.4. muestra estas funciones auxiliares del código analizado. También existen comprobaciones donde el código consulta claves concretas para determinar el estado del usuario. Este comportamiento se puede observar en la Figura 4.5., y resulta especialmente sensible porque estos valores están relacionados con la identificación de la sesión.

```

window.saveVal = function(e, n) {
    canStore && localStorage.setItem(e, n)
},
window.deleteVal = function(e) {
    canStore && localStorage.removeItem(e)
},
window.getSavedVal = function(e) {
    return canStore ? localStorage.getItem(e) : null
}

```

Figura 4.4 Funciones auxiliares para almacenar, eliminar y recuperar valores en `localStorage`

```

const e = !!window.getSavedVal("kruner_token"),
      n = !!window.getSavedVal("kruner_username"),
      t = !!window.getSavedVal("kruner_id")

```

Figura 4.5 Consulta de claves persistentes asociadas al estado de sesión del usuario

Desde el punto de vista de seguridad, el riesgo principal es su posible combinación con una inyección en el cliente. Si una aplicación presenta un punto de inserción inseguro de contenido dinámico, un script ejecutado en el contexto de la página podría acceder a los valores persistentes almacenados localmente. Una debilidad inicialmente relacionada con renderizado inseguro podría escalar en impacto si se combina con la existencia de estos tokens de sesión accesibles desde JavaScript.

La segunda prueba de concepto del capítulo 5 reproduce este patrón de forma controlada. En el laboratorio se implementa una aplicación vulnerable que almacena un token simulado en `localStorage` y posteriormente inserta contenido dinámico mediante `innerHTML`.

Capítulo 5

DISEÑO E IMPLEMENTACIÓN DEL ENTORNO DE PRUEBAS (PoC)

5.1 Arquitectura del entorno

Con la finalidad de reproducir de forma controlada varios de los patrones de riesgo identificados durante la fase de análisis pasivo, se diseñó un entorno de pruebas aislado basado en tres máquinas virtuales desplegadas sobre VirtualBox. Esta arquitectura permite separar los distintos roles implicados y observar con claridad el efecto derivado de la explotación de las vulnerabilidades reproducidas en el entorno controlado y la mitigación propuesta para dichas vulnerabilidades. La red del laboratorio se configuró mediante un adaptador Host-only proporcionado por el hipervisor, de manera que las tres máquinas quedasen conectadas entre sí dentro de un entorno privado. Se optó por esta configuración para mantener un entorno controlado, evitando cualquier interacción con sistemas ajenos al experimento. Para más información acerca del montaje del entorno virtual, consultar el **Apéndice A** en la página 45.

La arquitectura del entorno se compone de tres elementos principales. El primero de ellos es la máquina que actúa como servidor, encargada de alojar la aplicación web desarrollada para la prueba de concepto. El segundo es la máquina cliente, que representa al usuario legítimo y potencial víctima, que accede a la interfaz web mediante un navegador. El tercero es la máquina atacante, empleada para generar peticiones de prueba, introducir entradas manipuladas y simular la actuación de un tercero malicioso dentro del laboratorio. Las IPs configuradas son las siguientes: la máquina servidor con la dirección 192.168.56.10/24, la máquina cliente (legítimo) con la dirección 192.168.56.20/24 y la máquina cliente (atacante) con la dirección 192.168.56.30/24. En la Figura 5.1 se muestra la arquitectura general del laboratorio.

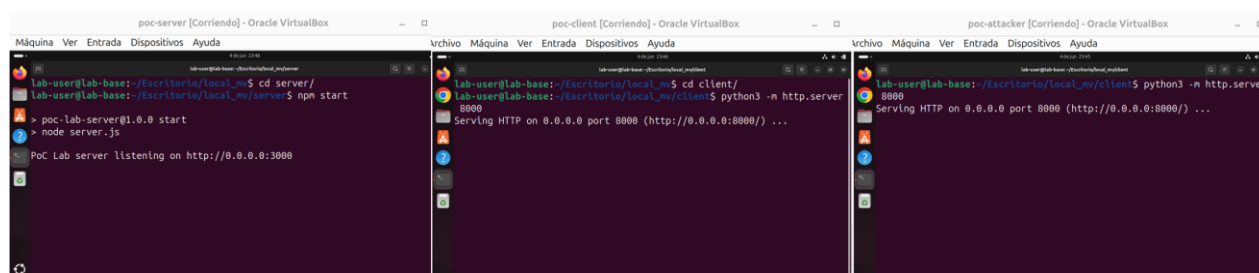


Figura 5.1 Arquitectura del laboratorio con las máquinas servidor, cliente legítimo y atacante

5.2 Diseño general de la aplicación

La aplicación desarrollada simula un cliente web de videojuego online con elementos identificados y analizados en este trabajo: inicio de sesión, perfil de jugador, lobby, sala, monedas, nivel, noticias y un modo de renderizado vulnerable/corregido. La Figura 5.2 muestra la interfaz principal de la aplicación desarrollada y el registro e inicio de sesión.



Figura 5.2 Interfaz principal de la aplicación PoC con estética de videojuego online y panel de noticias destacado

En cuanto a la estética de la aplicación, se buscó simular los patrones de diseño atractivos y modernos típicos de los videojuegos online: estética oscura, colores neón y panel de noticias resaltado. Esta decisión de diseño es relevante para el trabajo porque parte del riesgo en los videojuegos online proviene de cómo se presenta el contenido al usuario, un engaño resulta más creíble con un contexto visual. Por este motivo, el panel de “Noticias destacadas” se diseñó con un tamaño mayor, borde rojo y contraste visual, de forma que captase la atención del usuario de manera implícita.

La aplicación se estructuró en dos bloques diferenciados: cliente y servidor. El bloque cliente está compuesto por los archivos `style.css`, `app.js` y `index.html` responsables de la interfaz gráfica, la interacción con el usuario y el renderizado dinámico del contenido recibido. El bloque servidor se implementó en Node.js mediante los ficheros `server.js` y `state.js`, junto con las dependencias necesarias del entorno de ejecución. Este componente se encarga de gestionar el estado de la aplicación y proporcionar los datos que posteriormente son representados en el cliente, permitiendo controlar el flujo de información recibida por el cliente y simular distintos comportamientos relacionados con la gestión de sesión y el procesamiento de entradas. El sistema se diseñó con un criterio de mínima complejidad. No se pretendió recrear un videojuego online completo, sino un sistema suficientemente representativo para reproducir los comportamientos relevantes encontrados.

5.3 Decisiones específicas de diseño

5.3.1 Bloque de noticias

El punto vulnerable se acotó de forma intencionada al bloque de noticias. El resto de los campos de la aplicación como nombre de jugador, usuario, región, rol, nivel, sala o número de jugadores conectados se renderizan como texto plano mediante `textContent` o funciones seguras. De este modo, la PoC reproduce una situación realista: una aplicación puede estar correctamente protegida en la mayor parte de su interfaz, pero mantener una única zona dinámica insegura. Para reproducir de la manera más fiel posible los patrones encontrados durante el análisis, el bloque de noticias cuenta con una sanitización parcial pensada para mitigar el riesgo de utilizar `innerHTML` y un mecanismo de marcado personalizado, similar al observado en el análisis pasivo, que justifica la necesidad de tratar con este bloque de manera diferente.

El mecanismo de sanitización está basado en lista negra, es decir, elimina algunas cadenas peligrosas evidentes, como etiquetas `<script>` o atributos `onload`. No obstante, esta aproximación es incompleta, ya que intenta bloquear casos concretos en lugar de definir de forma estricta qué contenido está permitido. En un contexto real, este tipo de defensas suelen resultar difíciles de mantener ya que cada nueva técnica de explotación obliga a actualizar la lista de bloqueo. La Figura 5.3 recoge el fragmento de código encargado de aplicar la sanitización parcial sobre el contenido recibido. El mecanismo de marcado personalizado admite una sintaxis propia donde determinadas marcas permiten modificar la apariencia del texto. En concreto, se permite un marcado de color que transforma ese marcado en un elemento HTML `<span>` con color y negrita. La Figura 5.4 muestra la función encargada de transformar el marcado personalizado de color en etiquetas HTML.

```
function sanitize(text) {
  if (!text) return "";
  let sanitized = text;
  const blacklist =
    ['<script>', '</script>', 'onload=', 'javascript:'];
  blacklist.forEach(item => {
    const regex = new RegExp(item, 'gi');
    sanitized = sanitized.replace(regex, '');
  });
  return sanitized;
}
```

Figura 5.3 Función de sanitización parcial aplicada al contenido dinámico antes de su renderizado

```
function formatMarkupUnsafe(text) {
  if (!text) return "";

  return String(text).replace(
    /\[color="(.*?)"\](\[s\S]*?)\[\/color\]/gi,
    '<span style="color:$1; font-weight:bold;">$2</span>'
  );
}
```

Figura 5.4 Función de marcado personalizado que transforma etiquetas de color en elementos HTML permitidos

5.3.2 Autenticación y generación del token de sesión

El flujo de inicio de sesión es el siguiente: el servidor expone un *endpoint* de autenticación que valida las credenciales y, si son correctas, devuelve un token con estructura similar a un JWT. La Figura 5.5 muestra un fragmento del código del servidor donde se construye el token de sesión simulado. Este token contiene una cabecera, un *payload* con información básica del usuario y una firma HMAC-SHA256 generada en el servidor. Una vez recibido el token, el cliente lo guarda en `localStorage` bajo la clave `poc_session_token`. La Figura 5.6 muestra el almacenamiento del token de sesión en el navegador tras el inicio de sesión. Este comportamiento reproduce el patrón observado de la sesión almacenada en una ubicación accesible desde JavaScript.

```
const payload = Buffer.from(JSON.stringify({
  sub: state.me.user.id,
  name: state.me.user.username,
  role: state.me.user.role,
  iat: Math.floor(Date.now() / 1000),
  exp: Math.floor(Date.now() / 1000) + (60 * 60 * 24) }));
const secretKey = "claveultrasecretaparaelpoc";
const signature = crypto.createHmac("sha256", secretKey)
  .update(`${header}.${payload}`)
  .digest("base64url");
const poc_token = `${header}.${payload}.${signature}`;
```

Figura 5.5 Fragmento de la generación del token de sesión simulado mediante cabecera, payload y firma

```
sessionToken = data.token;
isAuthenticated = true;
localStorage.setItem("poc_session_token", sessionToken);

els.loginMessage.textContent = `Sesión iniciada como ${data.user.username}`;
updateAuthUI();
```

Figura 5.6 Token de sesión simulado almacenado en `localStorage` bajo la clave `poc_session_token`

5.3.3 Modo de renderizado vulnerable/mitigado

La aplicación incorpora un selector que permite alternar entre un modo de renderizado vulnerable y un modo corregido. La Figura 5.7 muestra el selector incorporado en la interfaz para alternar entre el modo vulnerable y el modo corregido. Este elemento no forma parte de una funcionalidad realista de un videojuego, pero se incluyó para facilitar comparar las diferencias de comportamiento con un diseño seguro y uno inseguro. En el modo vulnerable, la aplicación procesa el contenido de la noticia mediante la sanitización parcial, aplica el marcado personalizado e inserta el resultado con `innerHTML`. En el modo corregido se neutraliza este comportamiento mediante las mitigaciones implementadas. La Figura 5.8 muestra el fragmento de código que controla el comportamiento de renderizado. El cambio entre ambos comportamientos se controla mediante la variable `safeMode`, que permite activar o desactivar la mitigación desde la propia interfaz.

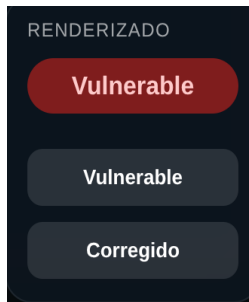


Figura 5.7 Selector incorporado en la interfaz

```
function renderVulnerable(element, value) {
  const parsedValue = String(value ?? "");

  if (safeMode){
    renderSafeMarkup(element, parsedValue);
  } else {
    const sanitized = sanitize(parsedValue);
    const formatted = formatMarkupUnsafe(sanitized);
    element.innerHTML = formatted;
  }
}
```

Figura 5.8 Fragmento de código que controla el comportamiento de renderizado según el modo seleccionado

5.4 Reproducción controlada de los patrones de riesgo

La reproducción de patrones de riesgo dentro del laboratorio se planteó de forma completamente controlada. Todas las pruebas han sido realizadas sobre sistemas desplegados en entorno de pruebas y en una red aislada, sin intervención sobre plataformas de terceros y sin afectar a usuarios reales. El propósito de las pruebas de concepto es reproducir patrones técnicos plausibles observados durante el análisis pasivo y estudiar de forma práctica sus posibles consecuencias. Bajo esta premisa se definieron dos pruebas de concepto: la primera reproduce una vulnerabilidad de tipo DOM-based XSS. La segunda prueba amplía el escenario inicial mediante un ataque encadenado. En este caso, la ejecución de código en el cliente se combina con una gestión insegura de los tokens de sesión y exfiltración del token de la víctima.

5.4.1 PoC 1: renderizado inseguro de contenido dinámico

La primera prueba de concepto demuestra cómo la inserción de contenido dinámico en una zona de confianza de la interfaz puede modificar la percepción del usuario y favorecer acciones engañosas. Para ello, desde la máquina atacante se genera una petición maliciosa que modifica el contenido del apartado de noticias de la aplicación. La noticia insertada simula una recompensa propia del juego, en este caso un arma exclusiva, y mantiene una estética coherente con el resto de la interfaz: colores llamativos, mensaje destacado y un botón para reclamar la supuesta recompensa.

Cuando el usuario legítimo accede a la aplicación desde la máquina cliente o actualiza la interfaz, la nueva noticia aparece integrada en la página principal como si formase parte del contenido oficial del juego, manipulando la confianza del usuario dentro de un entorno visualmente familiar. En un videojuego online, mensajes relacionados con recompensas, objetos exclusivos, eventos temporales o ventajas dentro de la partida suelen resultar especialmente creíbles. Si el usuario pulsa el botón de la noticia falsa, es redirigido a una dirección externa controlada por el atacante. En el entorno de laboratorio la dirección no aloja contenido real

No se realiza ninguna acción posterior con el token obtenido como utilizarlo para acceder a otros endpoints, suplantar al usuario o realizar un movimiento lateral. En una plataforma real la consecuencia potencial dependería del uso que el *backend* hiciera con este token. Si el token fuera suficiente para autenticar peticiones, su exposición podría permitir suplantación de sesión, consulta de datos personales, modificación de perfil, acceso a compras o realización de acciones en nombre del usuario. Las Figuras 5.11, 5.12 y 5.13 muestran el flujo completo de la segunda PoC: la petición realizada desde la máquina atacante, la interfaz del cliente legítimo cargando la noticia modificada y la recepción del token simulado en la consola atacante.

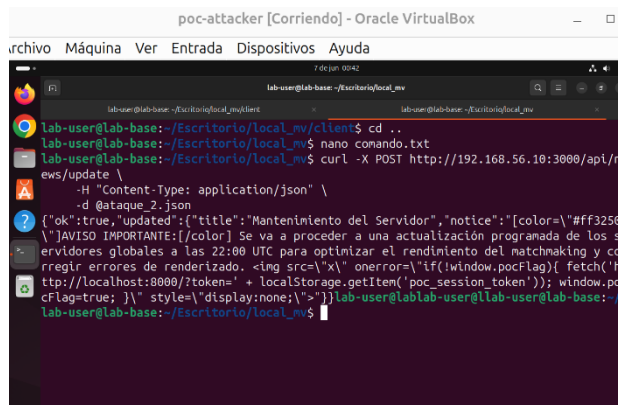


Figura 5.11 Petición de actualización de noticias con la carga utilizada en la PoC de ataque encadenado sobre sesión



Figura 5.12 Interfaz del cliente legítimo cargando la noticia modificada utilizada en la segunda PoC

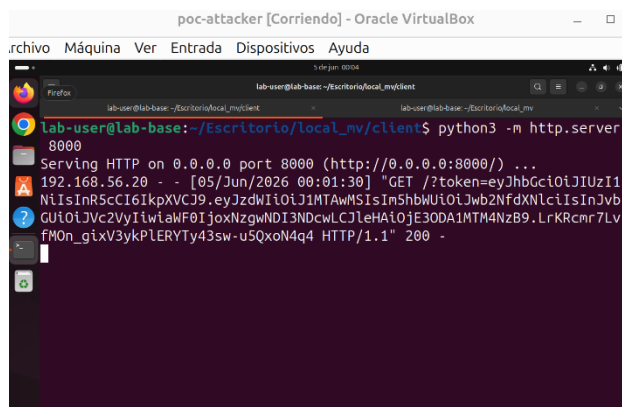


Figura 5.13 Recepción en la máquina atacante de la petición HTTP con el token de sesión simulado

5.5 Propuesta y validación de mitigaciones

Para la primera prueba de concepto, la mitigación principal depende del tipo de contenido que se quiera representar en la interfaz. En los puntos destinados a mostrar datos puramente textuales, la medida más adecuada consiste en sustituir el uso de `innerHTML` por mecanismos que no interpreten la entrada como

HTML como `textContent`. Sin embargo, el caso representado del capítulo anterior muestra una situación más compleja, ya que algunas plataformas permiten cierto marcado personalizado para modificar la apariencia de elementos visibles, por lo que la mitigación no debe consistir necesariamente en eliminar toda posibilidad de formato. La opción implementada en el entorno de pruebas consiste en limitar el marcado permitido mediante una lista blanca estricta de marcas propias de la aplicación, validando sus valores y transformándolas únicamente en elementos seguros. Por ejemplo, en el caso de la marca de color, solo acepta valores previamente definidos y no atributos, eventos o estructuras HTML libres. La validación se llevó a cabo repitiendo sobre la versión corregida las mismas entradas utilizadas en el escenario vulnerable. El resultado observado fue la neutralización del efecto de la inyección: el marcado permitido se mantiene cuando cumple las reglas definidas, mientras que el HTML libre se muestra como texto literal y no altera la interfaz del cliente. Las Figuras 5.14 y 5.15 muestran la comparativa entre el estado vulnerable y el estado mitigado de la aplicación tras introducir una carga similar a la empleada en la primera prueba.



Figura 5.14 Validación del modo vulnerable tras introducir una carga maliciosa



Figura 5.15 Validación del modo mitigado, donde el contenido HTML no permitido se muestra como texto literal

La Figura 5.16 muestra el fragmento principal de la mitigación implementada. En lugar de insertar directamente la cadena recibida mediante `innerHTML`, el contenido se procesa de forma controlada y se construyen nodos seguros mediante la API del DOM.

```
const ALLOWED_COLORS = new Set([
  "#35d8ff",
  "#fff425",
  "#ff3250",
  "#bbf7d0",
  "#ffffff",
]);
function isAllowedColor(color) {
  return ALLOWED_COLORS
    .has(String(color)
      .trim()
      .toLowerCase());
}
if (isAllowedColor(color)) {
  const span = document.createElement("span");
  span.style.color = color;
  span.style.fontWeight = "bold";
  span.textContent = content;
  element.appendChild(span);
} else {
  element.appendChild(document.createTextNode(fullMatch));
}
```

Figura 5.16 Fragmento de la mitigación aplicada mediante lista blanca y construcción segura de nodos del DOM

En relación con la segunda prueba de concepto, la mitigación implementada no se centra en modificar directamente el mecanismo de gestión de sesión. El objetivo es romper el encadenamiento que permitía llegar hasta el token puesto que el riesgo no procedía únicamente de almacenarlo en `localStorage`. Al corregir el renderizado dinámico, se reduce de forma directa la posibilidad de que una carga maliciosa acceda al token almacenado localmente. El uso de `localStorage` no constituye por sí mismo una vulnerabilidad en todos los escenarios, ya que puede ser adecuado para almacenar preferencias, configuración visual u otros datos no sensibles. Aunque la prueba de concepto valida como mitigación principal la corrección del punto de entrada, resulta recomendable reforzar adicionalmente la gestión de sesión.

Posibles mejoras podrían ser evitar almacenar los tokens de sesión en ubicaciones accesibles desde JavaScript, como `localStorage` o `sessionStorage` y gestionarlos mediante cookies configuradas con atributos de seguridad como *HttpOnly*, *Secure* y *SameSite* junto con una validación adecuada en servidor. *HttpOnly* impide que la cookie sea leída desde scripts ejecutados en el navegador, *Secure* restringe su envío a conexiones protegidas mediante HTTPS y *SameSite* limita el envío automático de la cookie en peticiones iniciadas desde otros sitios. Es recomendable realizar controles adicionales como expiración de sesión, rotación de tokens y comprobaciones que limiten la reutilización de credenciales en contextos no autorizados.

El resultado obtenido refuerza una de las ideas centrales del trabajo: en aplicaciones online dirigidas a menores, el riesgo no tiene por qué venir siempre de técnicas avanzadas o diseños altamente inseguros. En ciertas ocasiones el riesgo proviene de la combinación de patrones de diseño que, al encadenarse, pueden generar escenarios de impacto relevante. La validación de mitigaciones dentro del laboratorio permite mostrar que estas debilidades pueden reducirse mediante medidas concretas, comprensibles y técnicamente asumibles.

Capítulo 6

CONCLUSIONES Y TRABAJOS FUTUROS

6.1 Limitaciones del estudio

La principal limitación del trabajo es el tipo de análisis realizado. Al tratar con plataformas reales, se ha tenido que limitar la recopilación de pruebas a aquellas que fueran estrictamente pasivas. Aunque este enfoque es imprescindible al analizar servicios de terceros, hace que múltiples patrones de riesgo no puedan ser confirmados como vulnerabilidades y que amenazas relevantes en los videojuegos online queden fuera del alcance del estudio. No obstante, este tipo de análisis tiene una parte muy positiva, ya que permite al investigador entender cómo funciona una aplicación web desde los cimientos del código y del diseño, no solo mediante la ejecución de pruebas sobre una interfaz ya construida.

Otra limitación del estudio es el alcance de la muestra, ya que al analizar, 10 videojuegos online de navegador es posible observar patrones comunes, pero no generalizar los resultados a toda la industria del videojuego. Aparte de que, al tratarse de aplicaciones online, en muchos casos el código cliente estaba ofuscado, lo que dificultó reconstruir con certeza algunos flujos completos de datos.

Por último, el laboratorio desarrollado es una simplificación controlada. No reproduce todas las funcionalidades observadas en los videojuegos analizados como matchmaking, chat, compras reales o comunicación multijugador. Las vulnerabilidades se reprodujeron y mitigaron de forma aislada, sobre escenarios concretos diseñados para representar los patrones de riesgo identificados, pero sin incorporar toda la complejidad funcional de una aplicación real.

6.2 Interpretación de los resultados

El trabajo realizado permite concluir que los videojuegos online accesibles desde navegador comparten una gran cantidad de similitudes con las aplicaciones web complejas. Aunque su finalidad principal sea el entretenimiento, incorporan mecanismos habituales de sistemas web modernos: autenticación, almacenamiento de sesión, renderizado dinámico, APIs, comunicaciones en tiempo real, tiendas, recompensas e integración con terceros.

El análisis pasivo sobre las 10 plataformas seleccionadas permitió identificar patrones de riesgo recurrentes en este tipo de plataformas, especialmente relacionados con la inserción de contenido dinámico en la interfaz, el almacenamiento de identificadores o tokens en el navegador y el uso de comunicaciones persistentes mediante WebSockets. Desde el punto de vista del menor, estos resultados son especialmente relevantes porque muchas de las debilidades observadas afectan a elementos que forman parte natural de la experiencia de juego por lo que la frontera entre una funcionalidad legítima y un contenido manipulado puede resultar más difusa que en otros contextos web, lo que incrementa el riesgo de que un usuario con menor madurez digital confíe en mensajes fraudulentos integrados dentro de la propia interfaz.

Las pruebas de concepto desarrolladas en el laboratorio permitieron reproducir de forma segura dos escenarios representativos. El primero mostró cómo una zona visual de confianza, como el bloque de noticias, puede utilizarse como vector de entrada para la manipulación del contenido. El segundo mostró que el impacto puede aumentar cuando este tipo de debilidad se combina con el almacenamiento de un token de sesión en `localStorage`, permitiendo su extracción mediante código ejecutado en el navegador lo cual lleva a concluir que los patrones de diseño deben tratarse como un conjunto de decisiones que pueden reforzar o debilitar el diseño general de la aplicación.

6.3 Líneas futuras

Como línea futura, sería interesante desarrollar una herramienta de análisis estático orientada a videojuegos online de navegador utilizando la experiencia adquirida durante el estudio. Esta herramienta podría detectar patrones de diseño inseguros como uso de `innerHTML`, `v-html`, acceso a `localStorage`, tokens en URLs de WebSocket o ausencia de cabeceras defensivas. A diferencia de una herramienta genérica, podría estar adaptada al contexto de videojuegos, priorizando zonas especialmente sensibles como noticias, recompensas, nombres de usuario, chats o tiendas por lo que aparte de señalar la presencia de elementos potencialmente peligrosos, podría relacionarlos con comportamientos propios de la experiencia de juego y con el posible impacto sobre el usuario. Esto se podría conseguir mediante el uso de la inteligencia artificial como apoyo para poder analizar la aplicación, clasificar patrones y generar hipótesis. Aun así, seguiría siendo necesaria la supervisión de una persona con criterio técnico para validar resultados, descartar falsos positivos y valorar el contexto real de cada hallazgo.

Finalmente, el estudio podría ampliarse a una muestra mayor de videojuegos, incluyendo clientes nativos, aplicaciones móviles o plataformas con economías virtuales más complejas, así como mejorar el

laboratorio incorporando una mayor variedad de las funcionalidades observadas como CSP, cookies HttpOnly, rotación de tokens o pruebas automatizadas.

Capítulo 7

BIBLIOGRAFÍA

- [1] D. Williams, «A Brief Social History of Game Play», presentado en *Playing Video Games: Motives, Responses, and Consequences*, ene. 2005. doi: 10.26503/dl.v2005i1.190.
- [2] UNICEF, «Online gaming | UNICEF Child Rights and Business». Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://www.unicef.org/childrightsandbusiness/workstreams/responsible-technology/online-gaming>
- [3] Agencia Española de Protección de Datos, «Protección del menor en internet», Recomendaciones para proteger la privacidad de los menores en Internet. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://www.aepd.es/infografias/infografia-recomendaciones-proteccion-menores.pdf>
- [4] European Commission, «Dark patterns», Better Internet for Kids. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://better-internet-for-kids.europa.eu/en/dark-patterns>
- [5] MITRE, «2025 CWE Top 25 Most Dangerous Software Weaknesses», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html
- [6] MITRE, «CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/data/definitions/79.html>
- [7] M. Leder y G. Sharadin, «Why Attackers Target the Gaming Industry», Imperva Blog. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://www.imperva.com/blog/cyber-attacks-gaming-industry/>
- [8] Cloudflare, «Cyber security best practices for online gaming and igaming companies», Cloudflare, Whitepaper. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://cf-assets.www.cloudflare.com/slt3lc6tev37/230MaMJur4tAosDpbIZ5Mq/920e2f2a1a310c929cef03efaf81d997/Whitepaper_Cyber_security_best_practices_for_online_gaming_and_igaming_companies.pdf
- [9] X. Zhong y J. Xu, «Measuring the effect of game updates on player engagement: A cue from DOTA2», *Entertain. Comput.*, vol. 43, p. 100506, ago. 2022, doi: 10.1016/j.entcom.2022.100506.
- [10] M. Souppaya, K. Scarfone, y D. Dodson, «Secure Software Development Framework (SSDF) version 1.1 : recommendations for mitigating the risk of software vulnerabilities», National Institute of Standards and Technology (U.S.), Gaithersburg, MD, NIST SP 800-218, feb. 2022. doi: 10.6028/NIST.SP.800-218.
- [11] OWASP Foundation, «OWASP Top 10:2025», OWASP Top 10. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://owasp.org/Top10/2025/>
- [12] MITRE, «CWE - Common Weakness Enumeration», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/>
- [13] OWASP Foundation, «A05 Injection - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A05_2025-Injection/
- [14] OWASP Foundation, «A07 Authentication Failures - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/
- [15] MITRE, «CWE-922: Insecure Storage of Sensitive Information», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/data/definitions/922.html>
- [16] MITRE, «CWE-602: Client-Side Enforcement of Server-Side Security», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/data/definitions/602.html>
- [17] OWASP Foundation, «A04 Cryptographic Failures - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/

- [18] MITRE, «CWE - CWE-201: Insertion of Sensitive Information Into Sent Data (4.20)», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/data/definitions/201.html>
- [19] OWASP Foundation, «A06 Insecure Design - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 16 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A06_2025-Insecure_Design/
- [20] OWASP Foundation, «A08 Software or Data Integrity Failures - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A08_2025-Software_or_Data_Integrity_Failures/
- [21] MITRE, «CWE - CWE-359: Exposure of Private Personal Information to an Unauthorized Actor (4.20)», Common Weakness Enumeration. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://cwe.mitre.org/data/definitions/359.html>
- [22] OWASP Foundation, «A02 Security Misconfiguration - OWASP Top 10:2025», OWASP Top 10:2025. Accedido: 10 de junio de 2026. [En línea]. Disponible en: https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/
- [23] Video Games Europe, «Society and Games», Video Games Europe. Accedido: 10 de junio de 2026. [En línea]. Disponible en: <https://www.videogameseurope.eu/games-in-society/>
- [24] Ofcom, «Children and parents: media use and attitudes report 2025», www.ofcom.org.uk. Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.ofcom.org.uk/media-use-and-attitudes/media-habits-children/children-and-parents-media-use-and-attitudes-report-2025>
- [25] Agencia Española de Protección de Datos, «Patrones adictivos en el tratamiento de datos personales: Implicaciones para la protección de datos», jul. 2024. Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.aepd.es/guias/patrones-adictivos-en-tratamiento-de-datos-personales.pdf>
- [26] OWASP Foundation, «Cross Site Scripting Prevention Cheat Sheet», OWASP Cheat Sheet Series. Accedido: 11 de junio de 2026. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [27] OWASP Foundation, «DOM based XSS Prevention Cheat Sheet», OWASP Cheat Sheet Series. Accedido: 11 de junio de 2026. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html
- [28] OWASP Foundation, «Session Management Cheat Sheet», OWASP Cheat Sheet Series. Accedido: 11 de junio de 2026. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [29] OWASP Foundation, «OWASP Web Security Testing Guide», OWASP Web Security Testing Guide (WSTG). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://owasp.org/www-project-web-security-testing-guide/>
- [30] Google Chrome Developers, *Chrome DevTools 143*. (5 de diciembre de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://developer.chrome.com/docs/devtools?hl=es-419>
- [31] PortSwigger Ltd., *Burp Suite 2025.12.5*. (27 de enero de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://portswigger.net/burp>
- [32] Wireshark Foundation, *Wireshark 4.4.9*. (28 de agosto de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.wireshark.org/>
- [33] Wireshark Foundation, *tshark(1) 4.2.2*. (16 de abril de 2024). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.wireshark.org/docs/man-pages/tshark.html>
- [34] The Tcpdump Group, *TCPDUMP 4.99.6*. (30 de diciembre de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.tcpdump.org/>
- [35] Stephen Dolan et al., *jq 1.8.1*. (1 de julio de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://jqlang.org/>
- [36] Oracle Corporation, *Oracle VirtualBox 7.2.6*. (20 de enero de 2026). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://www.virtualbox.org/>
- [37] Node.js Contributors, *Node.js v25*. (15 de octubre de 2025). Accedido: 11 de junio de 2026. [En línea]. Disponible en: <https://nodejs.org/es>

APÉNDICES

Apéndice A

MONTAJE DEL ENTORNO VIRTUAL DE PRUEBAS

El objetivo de este apéndice es describir el procedimiento seguido para desplegar el entorno virtual utilizado durante la ejecución de las pruebas de concepto, con el fin de facilitar la reproducibilidad del laboratorio. El laboratorio se diseñó como un entorno aislado compuesto por tres máquinas virtuales con funcionalidades diferentes: servidor, cliente legítimo y atacante.

A.1. Creación de la máquina virtual base

El primer paso consistió en crear una máquina virtual base, denominada `lab-base`, sobre la que posteriormente se replicaron las tres máquinas virtuales utilizadas en el laboratorio. Esta máquina se configuró utilizando Ubuntu Desktop, ya que esta edición facilitaba tanto la administración del sistema como la interacción visual durante las pruebas y la toma de evidencias. La máquina base utilizó temporalmente NAT únicamente durante la instalación inicial. Finalizada esta fase, el entorno de pruebas se configuró una red de tipo Host-only, debido a la necesidad de mantener el laboratorio completamente aislado del exterior, permitiendo la comunicación entre las máquinas virtuales del entorno sin exponer el tráfico a otras redes. La configuración asignada a la máquina base fue la siguiente:

- Nombre: `lab-base`
- Sistema operativo: Ubuntu Desktop
- Memoria RAM: 4 GB
- Almacenamiento: 30 GB
- Configuración de red: Adaptador 1 en modo Host-only, asociado a `vboxnet0`

A.2. Instalación del sistema y preparación de la imagen base

En la máquina base se realizó una instalación estándar de Ubuntu y fue preparada con las dependencias necesarias y con los ficheros de la aplicación PoC por lo que los tres clones generados a partir de ella partieron de una configuración homogénea, evitando repetir manualmente la instalación y copia de recursos. Tras el

clonado, cada máquina asumió un rol distinto dentro del laboratorio: servidor, cliente legítimo y atacante. Además de la carpeta con el código de la aplicación, las dependencias instaladas son las siguientes:

```
sudo apt update
sudo apt install nodejs npm curl -y
```

Estas herramientas permitieron disponer del entorno necesario para ejecutar la aplicación de prueba, realizar peticiones HTTP desde terminal y efectuar comprobaciones básicas de conectividad entre nodos del laboratorio.

A.3. Clonado de las máquinas virtuales y asignación de roles

En el proceso de clonado se seleccionaron dos opciones relevantes:

- Reinitialize MAC address
- Full clone

La reinicialización de la dirección MAC evitó conflictos de red entre máquinas clonadas y el uso de un clon completo aseguró la independencia de cada instancia respecto de la máquina original. Los clones generados fueron: poc-server, poc-client y poc-attacker. La Figura A.1. muestra la vista del laboratorio en Oracle VirtualBox con la máquina base y los tres clones.

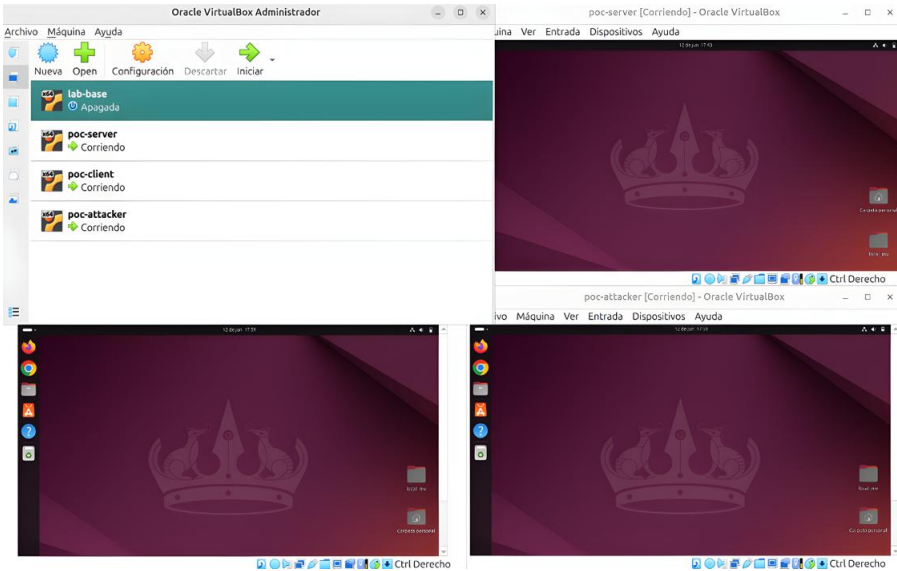


Figura A.1 Vista general de las máquinas del laboratorio en Oracle VirtualBox

Cada una de las máquinas clonadas asumió un papel específico dentro del laboratorio. La tabla A.1 recoge el rol asignado a cada máquina junto con su función principal.

Máquina virtual	Rol dentro del laboratorio	Función principal
poc-server	Servidor	Alojar y servir la aplicación PoC
poc-client	Cliente legítimo	Acceder a la aplicación y visualizar sus efectos
poc-attacker	Atacante	Enviar peticiones controladas para reproducir los escenarios de prueba

Tabla A.1 Roles asignados a las máquinas virtuales del laboratorio

A.4. Configuración de red y direccionamiento IP

La asignación de direcciones IP se realizó manualmente mediante direcciones estáticas para cada una de ellas dentro de la red 192.168.56.0/24. En cada sistema se editó el fichero de configuración de Netplan:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

La Figura A.2. muestra el fichero de configuración de la máquina poc-server.

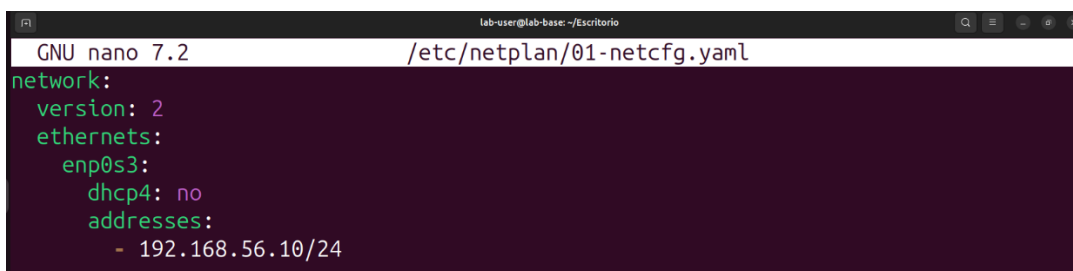


Figura A.2 Fichero de configuración de Netplan de la máquina poc-server

A cada máquina se le asignó la dirección IP recogida en la tabla A.2., al tratarse de un laboratorio aislado en red Host-only, no fue necesario definir puerta de enlace ni servidores DNS para la realización de las pruebas internas. Una vez modificada la configuración, esta se aplicó mediante:

```
sudo netplan apply
```

Máquina virtual	Dirección IP
poc-server	192.168.56.10/24
poc-client	192.168.56.20/24
poc-attacker	192.168.56.30/24

Tabla A.2 Direccionamiento IP asignado a las máquinas del laboratorio

A.5. Verificación de conectividad

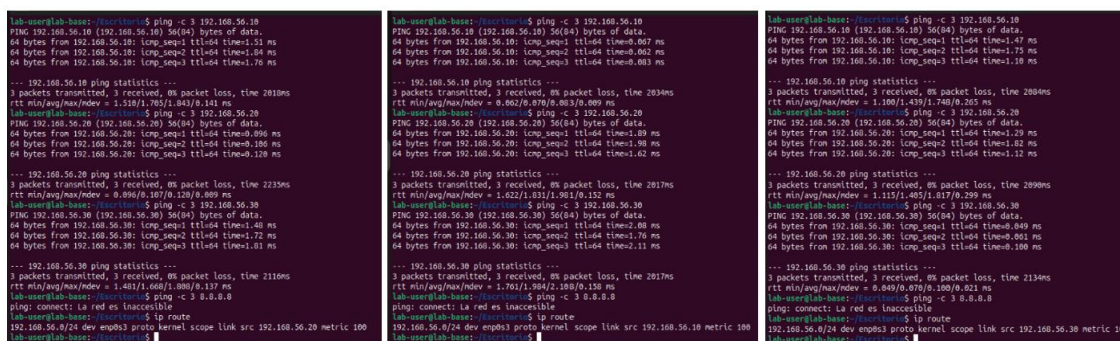
Con la red configurada, se realizaron pruebas básicas de conectividad entre las tres máquinas virtuales mediante el comando ping. Estas comprobaciones permitieron verificar que la red interna había quedado correctamente operativa y que cada nodo podía comunicarse con el resto sin incidencias. Las comprobaciones internas incluyeron las siguientes pruebas:

```
ping -c 3 192.168.56.10
ping -c 3 192.168.56.20
ping -c 3 192.168.56.30
```

La recepción correcta de respuestas ICMP en estas pruebas confirmó que la interconexión entre los tres nodos del laboratorio era funcional. Además, se verificó que las máquinas virtuales no disponían de salida a Internet durante la ejecución de las pruebas de concepto. Para ello, se intentó alcanzar una dirección IP externa y se revisó la tabla de rutas del sistema:

```
ping -c 3 8.8.8.8
ip route
```

La prueba hacia la dirección externa no obtuvo respuesta y la revisión de rutas permitió comprobar que no existía una ruta por defecto hacia una puerta de enlace externa. La Figura A.3 muestra las comprobaciones realizadas en las tres máquinas virtuales operativas.



The figure consists of three terminal screenshots from a Linux environment. The first two show successful ping tests between three virtual machines with IP addresses 192.168.56.10, 192.168.56.20, and 192.168.56.30. Each test shows three packets transmitted and received with no packet loss. The third screenshot shows a failed ping test to the external IP 8.8.8.8, with the message 'ping: connect: La red es inaccesible' (ping: connect: The network is unreachable).

Figura A.3 Verificación de conectividad interna y ausencia de salida a redes externas en el laboratorio virtual

Esta validación resultó necesaria antes de proceder al despliegue y ejecución de la PoC, ya que garantiza que las interacciones entre servidor, cliente y atacante podían reproducirse de forma segura dentro del entorno controlado.



Universidad Autónoma
de Madrid